

# ROS 101

Marco Faroni – Politecnico di Milano

4<sup>th</sup> Doctoral Summer School on Robotics and Intelligent Machines – DRIMS

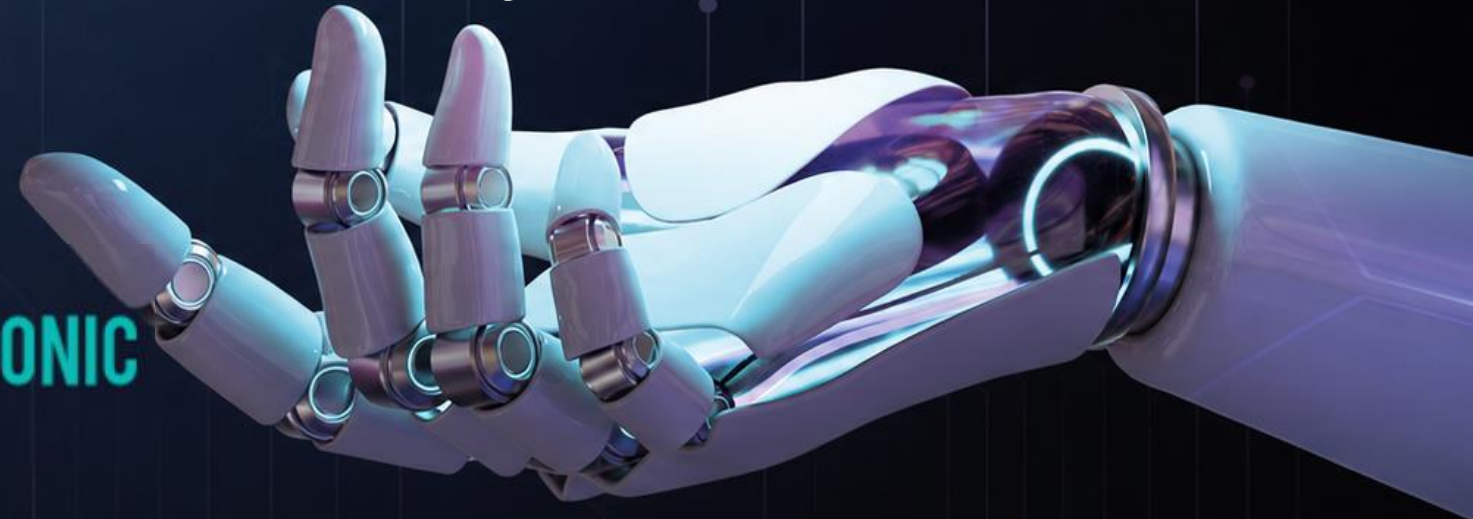
Gold Sponsor



Silver Sponsor



Bronze Sponsor



Aug. 30 – Sept. 6, 2026  
Poggio all'Agnello (Italy)



# About



Marco Faroni

Politecnico di Milano - DEIB



<https://merlin.deib.polimi.it/>

<https://github.com/marco-faroni>

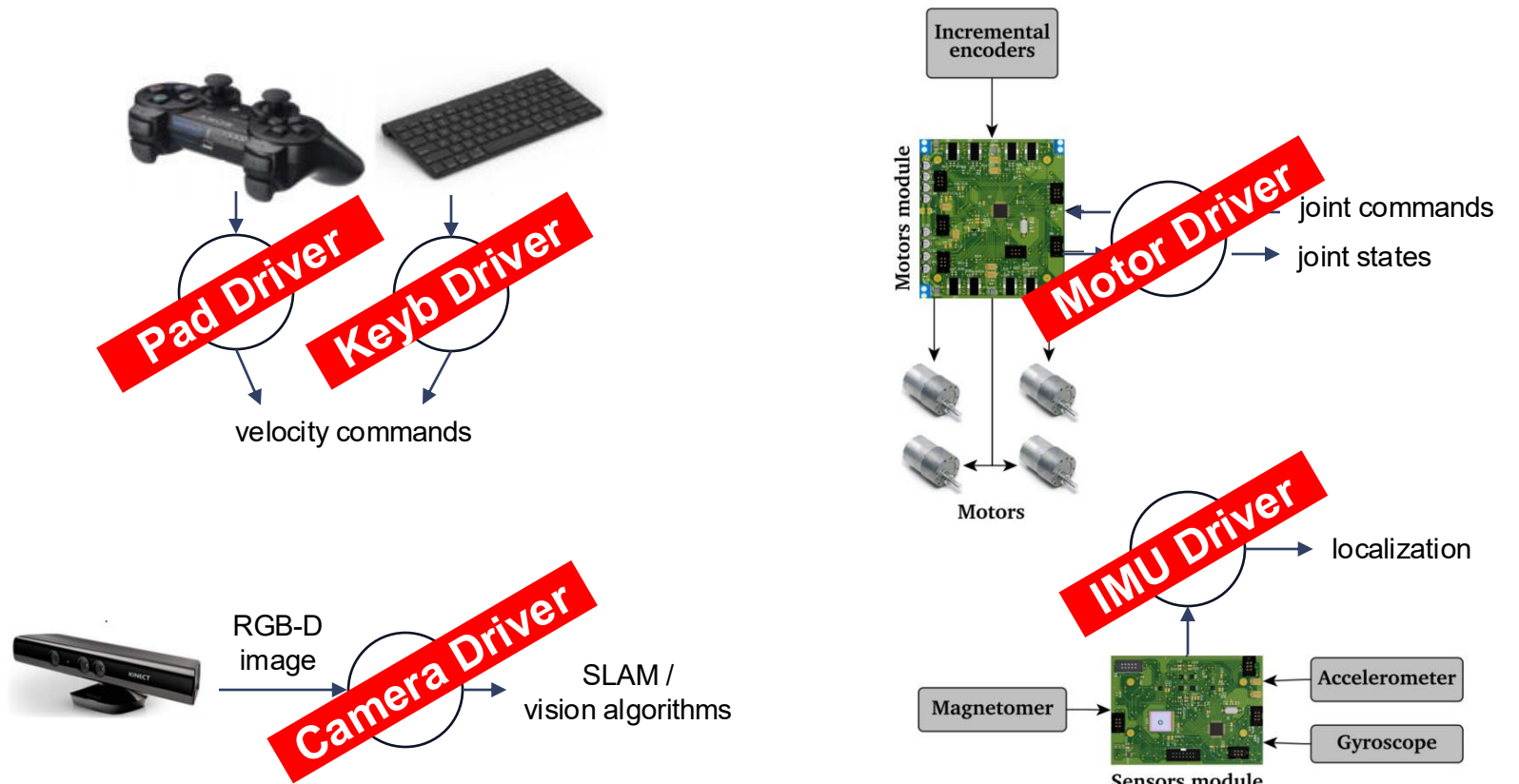
Research interests:

- Robotics manipulation
- Planning algorithms
- Human-robot collaboration

# Why ROS?



Four wheeled mobile robot with the autonomous navigation system in ros

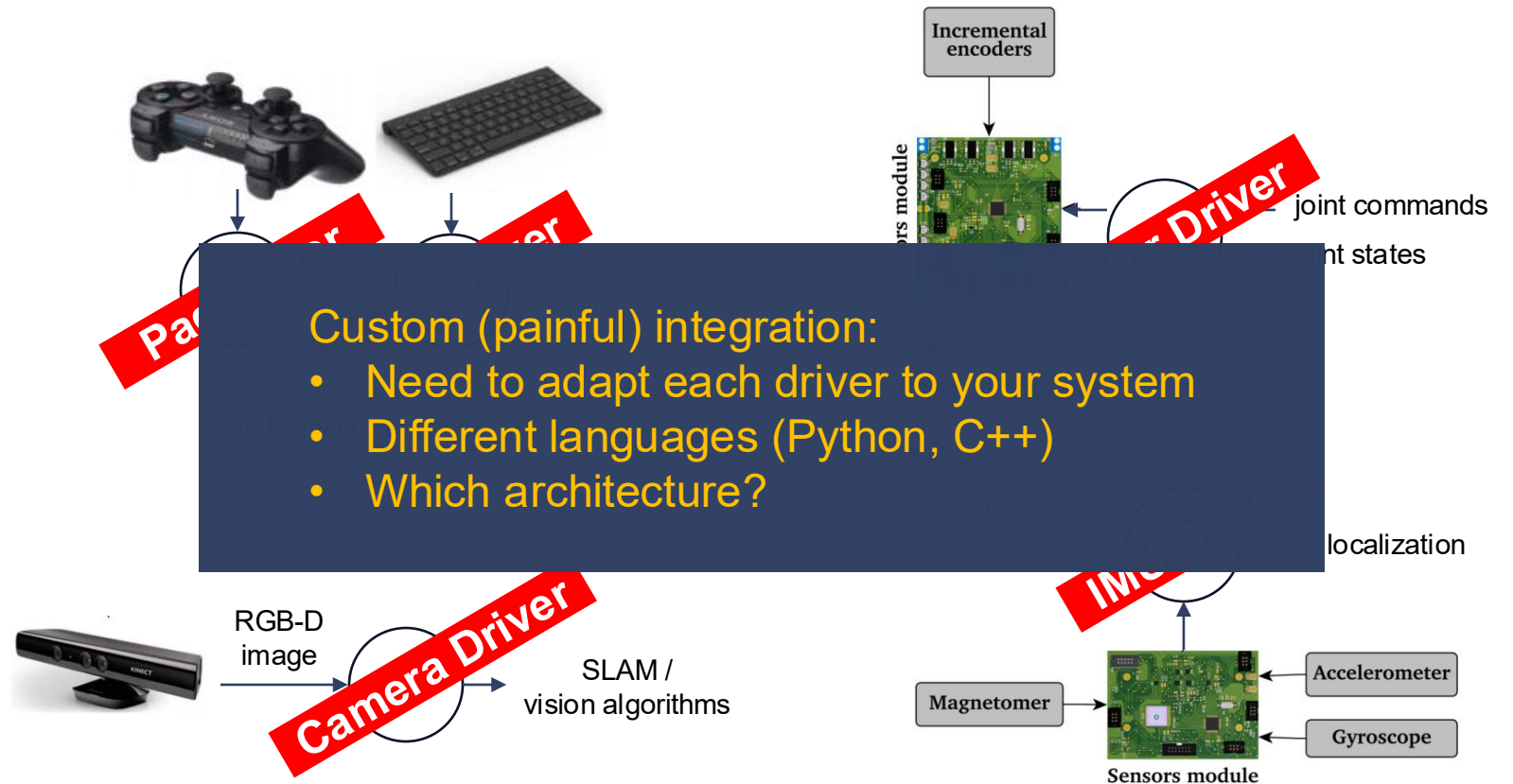




# Why ROS?



Four wheeled mobile robot with the autonomous navigation system in ros

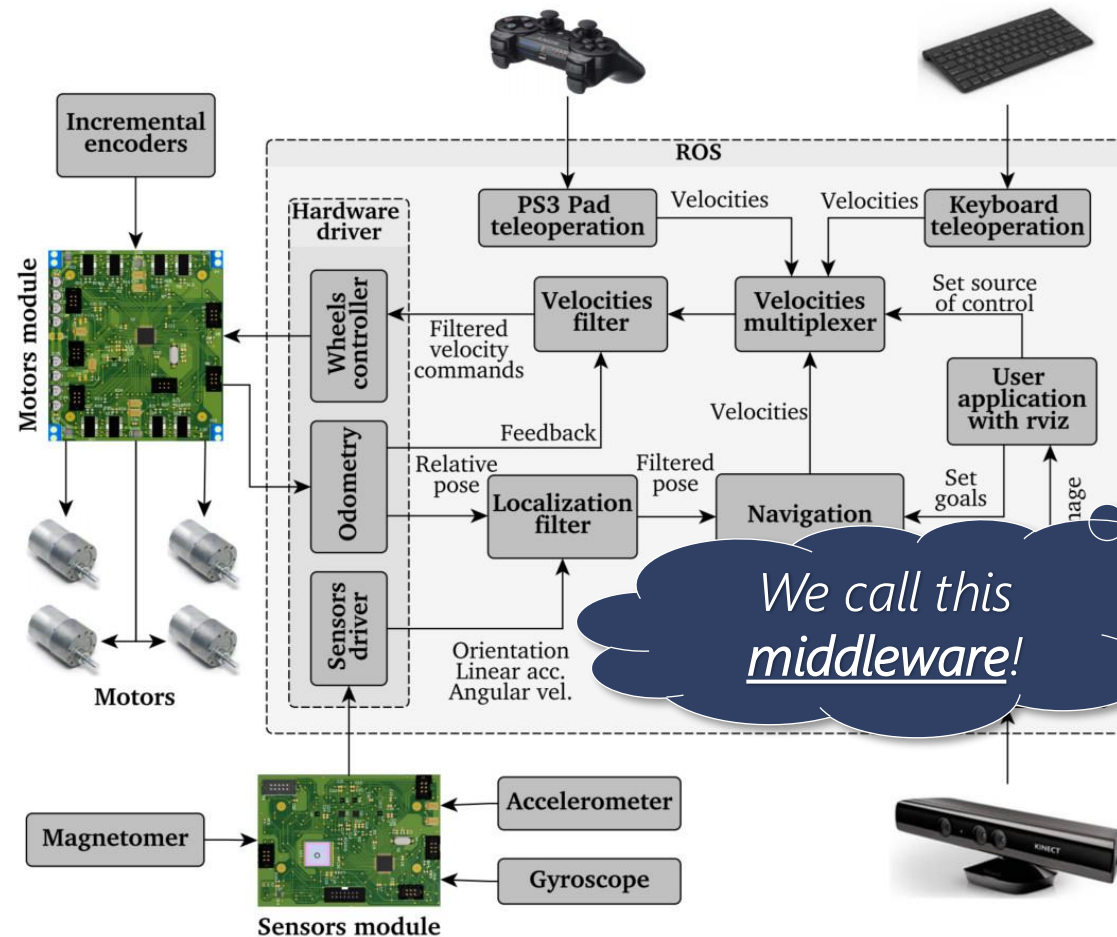


# Why ROS?

We need a common ground to stick everything together.



Four wheeled mobile robot with the autonomous navigation system in ros



Applications (nodes):

- Accept specific inputs
- Provide specific outputs

Connections:

Connect nodes

Different types dependings on needs

- Provide a standard architecture

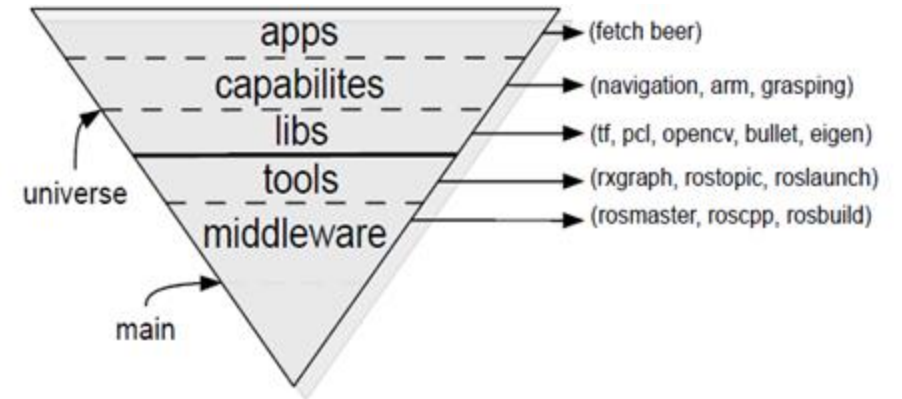
# Why ROS?

## Why a middleware?

- **Portability**: provides a common programming model regardless the programming language and the system architecture.
- **Reliability**: middleware are tested independently. They permit to develop robot controllers without considering the low-level details and using robust libraries.
- **Manage the complexity**: low-level aspects are handled by libraries and drivers inside the middleware. It (should) reduce(s) the programming error and decrease the development time.

# Why ROS?

- ROS = Robot Operating System
- However, not an OS, but mainly a middleware (+ ecosystem)





# Why ROS?

ROS has grown to include a large **community** of users worldwide

The community of developer is one of the most important characteristics of ROS

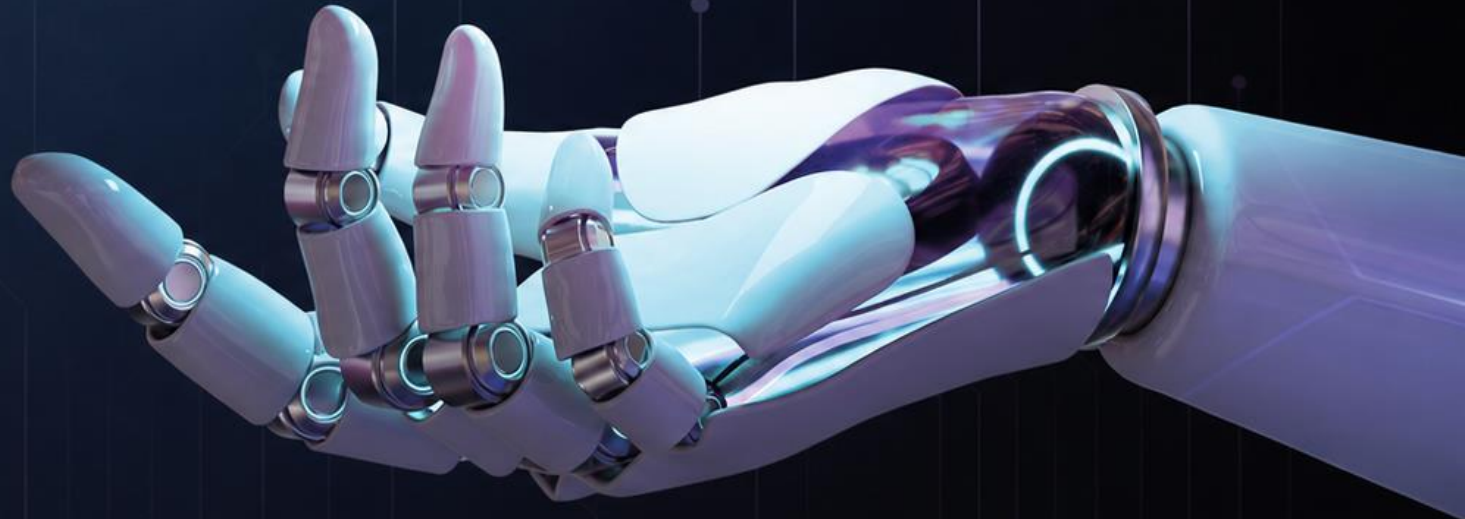
<https://discourse.openrobotics.org/t/2025-ros-metrics-report/52575>



| <input type="checkbox"/>            | Country           | Active users                 | New users                 |
|-------------------------------------|-------------------|------------------------------|---------------------------|
| <input checked="" type="checkbox"/> | Total             | 1,237,850<br>36.97% of total | 804,370<br>26.3% of total |
| <input type="checkbox"/>            | 1 China           | 363,439 (29.36%)             | 208,479 (25.92%)          |
| <input type="checkbox"/>            | 2 United States   | 201,282 (16.26%)             | 154,316 (19.18%)          |
| <input type="checkbox"/>            | 3 India           | 67,916 (5.49%)               | 45,576 (5.67%)            |
| <input type="checkbox"/>            | 4 Japan           | 65,352 (5.28%)               | 39,465 (4.91%)            |
| <input type="checkbox"/>            | 5 Germany         | 54,598 (4.41%)               | 33,340 (4.14%)            |
| <input type="checkbox"/>            | 6 South Korea     | 53,262 (4.3%)                | 35,886 (4.46%)            |
| <input type="checkbox"/>            | 7 Singapore       | 48,496 (3.92%)               | 16,786 (2.09%)            |
| <input type="checkbox"/>            | 8 Hong Kong       | 37,880 (3.06%)               | 20,889 (2.6%)             |
| <input type="checkbox"/>            | 9 Taiwan          | 26,422 (2.13%)               | 15,925 (1.98%)            |
| <input type="checkbox"/>            | 10 United Kingdom | 23,056 (1.86%)               | 15,558 (1.93%)            |
| <input type="checkbox"/>            | 11 Russia         | 22,952 (1.85%)               | 15,807 (1.97%)            |
| <input type="checkbox"/>            | 12 France         | 20,124 (1.63%)               | 12,867 (1.6%)             |
| <input type="checkbox"/>            | 13 Canada         | 19,740 (1.59%)               | 13,460 (1.67%)            |
| <input type="checkbox"/>            | 14 Türkiye        | 17,935 (1.45%)               | 12,201 (1.52%)            |
| <input type="checkbox"/>            | 15 Italy          | 17,734 (1.43%)               | 11,552 (1.44%)            |
| <input type="checkbox"/>            | 16 Spain          | 14,901 (1.2%)                | 9,357 (1.16%)             |
| <input type="checkbox"/>            | 17 Vietnam        | 14,691 (1.19%)               | 10,394 (1.29%)            |
| <input type="checkbox"/>            | 18 Brazil         | 11,857 (0.96%)               | 7,956 (0.99%)             |
| <input type="checkbox"/>            | 19 Australia      | 11,067 (0.89%)               | 7,277 (0.9%)              |
| <input type="checkbox"/>            | 20 Netherlands    | 10,608 (0.86%)               | 6,340 (0.79%)             |



# Installation



Aug. 30 – Sept. 6, 2026  
Poggio all'Agnello (Italy)



Cofinanziato  
dall'Unione europea



Ministero delle Imprese  
e del Made in Italy



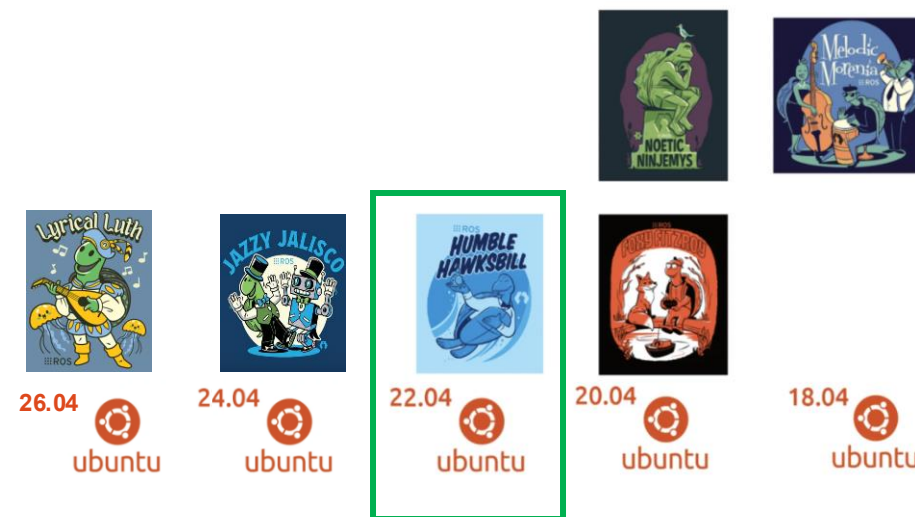
# ROS Distributions

## ROS Distro Share by Year

| Distro   | Oct 2024 | Oct 2025 YoY Change |
|----------|----------|---------------------|
| Noetic   | 22.18%   | 6.72% -15.46%       |
| Foxy     | 3.58%    | 1.73% -2.79%        |
| Galactic | 1.25%    | 0.94% -1.58%        |
| Humble   | 39.38%   | 48.53% +9.15%       |
| Iron     | 5.26%    | 0.70% -4.56%        |
| Jazzy    | 9.36%    | 24.45% +15.09%      |
| Kilted   | N/A      | 2.71% N/A           |
| Rolling  | 4.90%    | 3.46% -1.44%        |



ROS1 End-of-Life since May 31, 2025



Package downloads, by distro, as a percentage of all downloads from the ROS servers in October 2024, and 2025.



We will use ROS2 Humble on Ubuntu 22.04

# ROS on Docker

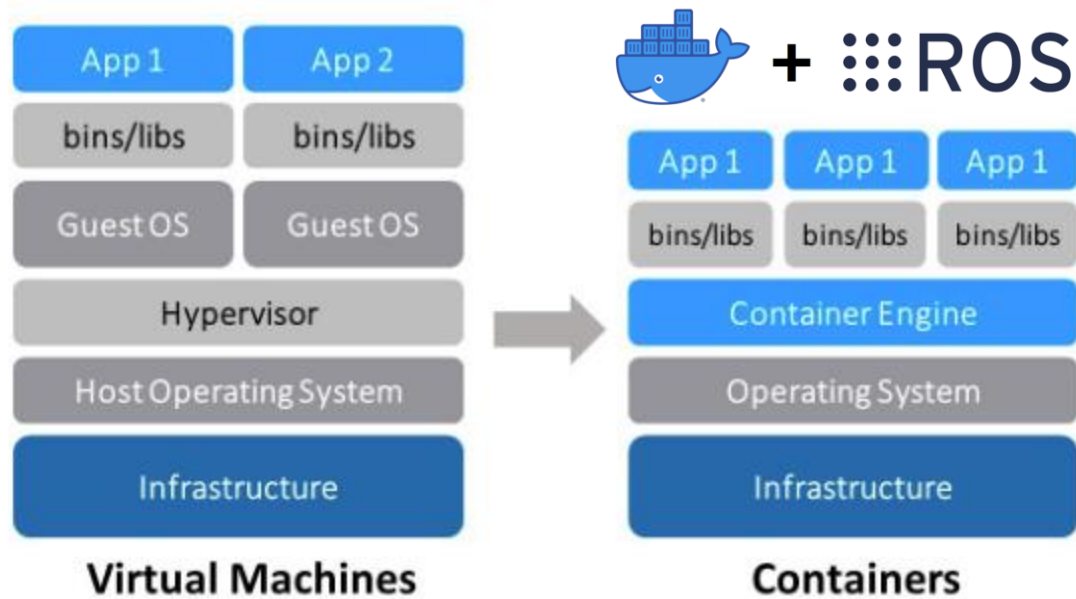


Figure: Virtual Machines Vs Containers (Docker)

- We use [Docker](#)
- You can see it as a lightweight VM\*

## Why Docker?

- Should work on any OS
- We all use the same docker image
- Self-contained, no dependencies issues
- Allows for Git-like management (Docker Hub)

**DRIMS2 Docker image is large,  
download it as soon as possible**

\*Not really, if you want to know more:

<https://www.freecodecamp.org/news/docker-vs-vm-key-differences-you-should-know/>

# DRIMS2 Docker Image

<https://github.com/AIRLab-POLIMI/DRIMS2-2026>



Simone  
Mentasti

AIRLab-POLIMI / DRIMS2-2026 Public

<> Code Issues Pull requests Actions Projects Security and quality Insights

main 1 Branch 0 Tags

Go to file Code

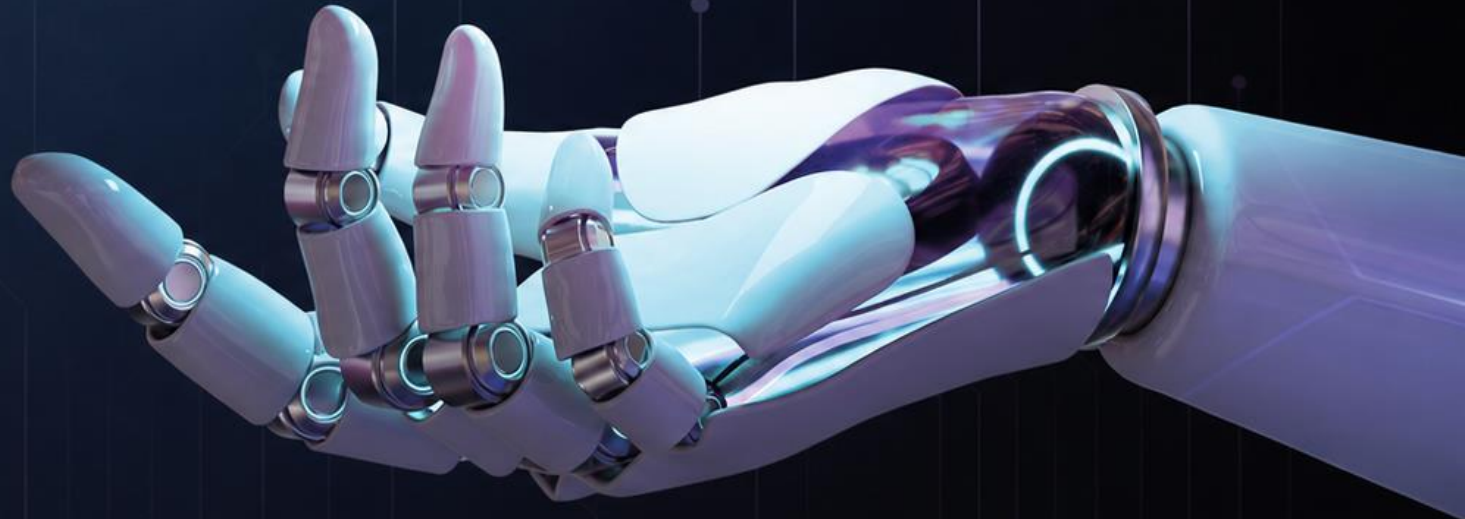
SMentasti updated all scripts to use 2026 image 7d049a6 · 3 weeks ago 72 Commits

|               |                                                                 |             |
|---------------|-----------------------------------------------------------------|-------------|
| bags          | added second mounted folder for bags files                      | 2 years ago |
| docker        | updated all scripts to use 2026 image                           | 3 weeks ago |
| drims_ws      | added example node                                              | last year   |
| windows_users | Added readme and files for windows user                         | last year   |
| .gitignore    | changed numpy version inside docker. added script to build i... | last year   |

DOWNLOAD AND TEST  
AS SOON AS POSSIBLE!



# ROS2 Computational Graph



Aug. 30 – Sept. 6, 2026  
Poggio all'Agnello (Italy)



Cofinanziato  
dall'Unione europea



Ministero delle Imprese  
e del Made in Italy



# ROS2 Computational Graph

## Core Entities:

- **Nodes** → independent processes doing computation
- **Topics** → publish/subscribe data streams coming from nodes
- **Services** → synchronous request/reply between nodes
- **Actions** → long-running goals with feedback
- **Parameters** → configurable settings for each node

**Graph:** All these entities form a dynamic *directed graph* of communication

For details: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html>

# Topics

## Example:

Terminal 1:

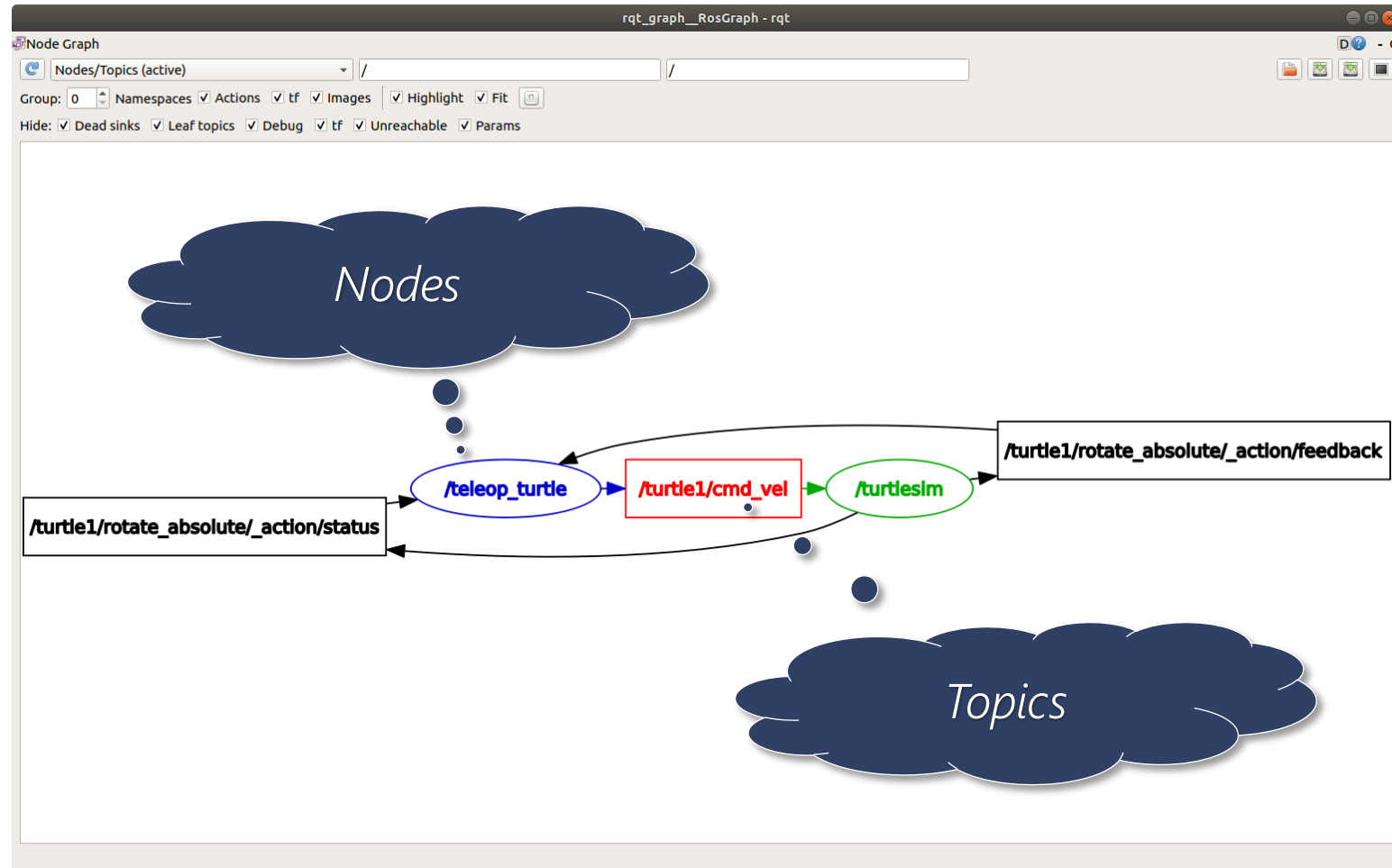
```
ros2 run turtlesim turtlesim_node
```

Terminal 2:

```
ros2 run turtlesim turtle_teleop_key
```

Terminal 3:

```
ros2 run rqt_graph rqt_graph
```



# Nodes

**Executable unit** of ROS:

- Scripts for Python
- Compiled source code for C++

Process that performs computation

Nodes exchange information via the graph

Meant to operate at fine-grained scale

A robot system is composed by various nodes

```
ros2 run <package_name> <node_name>
```

```
ros2 run turtlesim turtlesim_node
```

*Standard ROS  
tutorial demo*



# Nodes CLI tools

`ros2 run <pkg_name> <node_name>` – Run a node

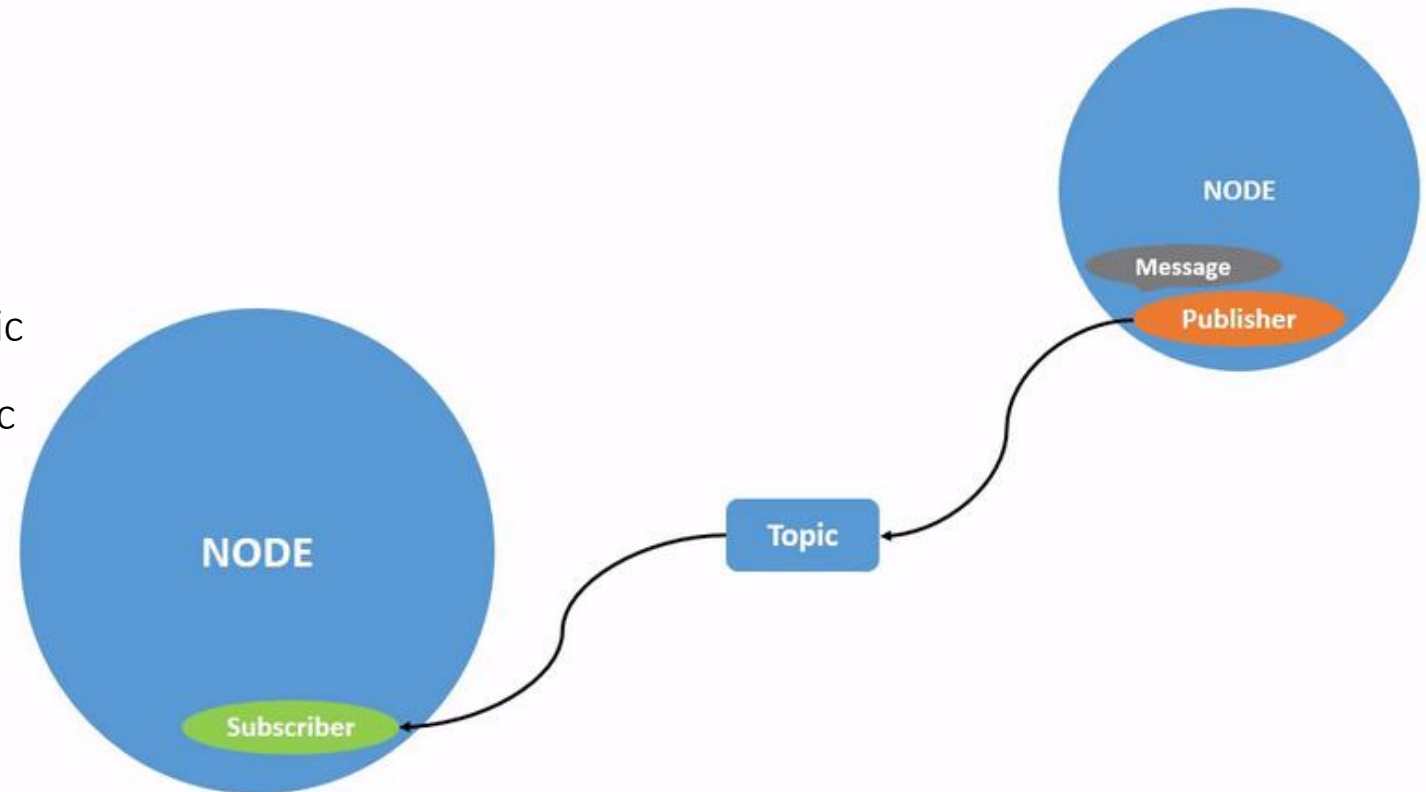
`ros2 node list` – Lists all currently active nodes

`ros2 node info -v <node_name>` – Displays the node's publishers, subscribers, services, and actions (-v stands for verbose)

# Topics

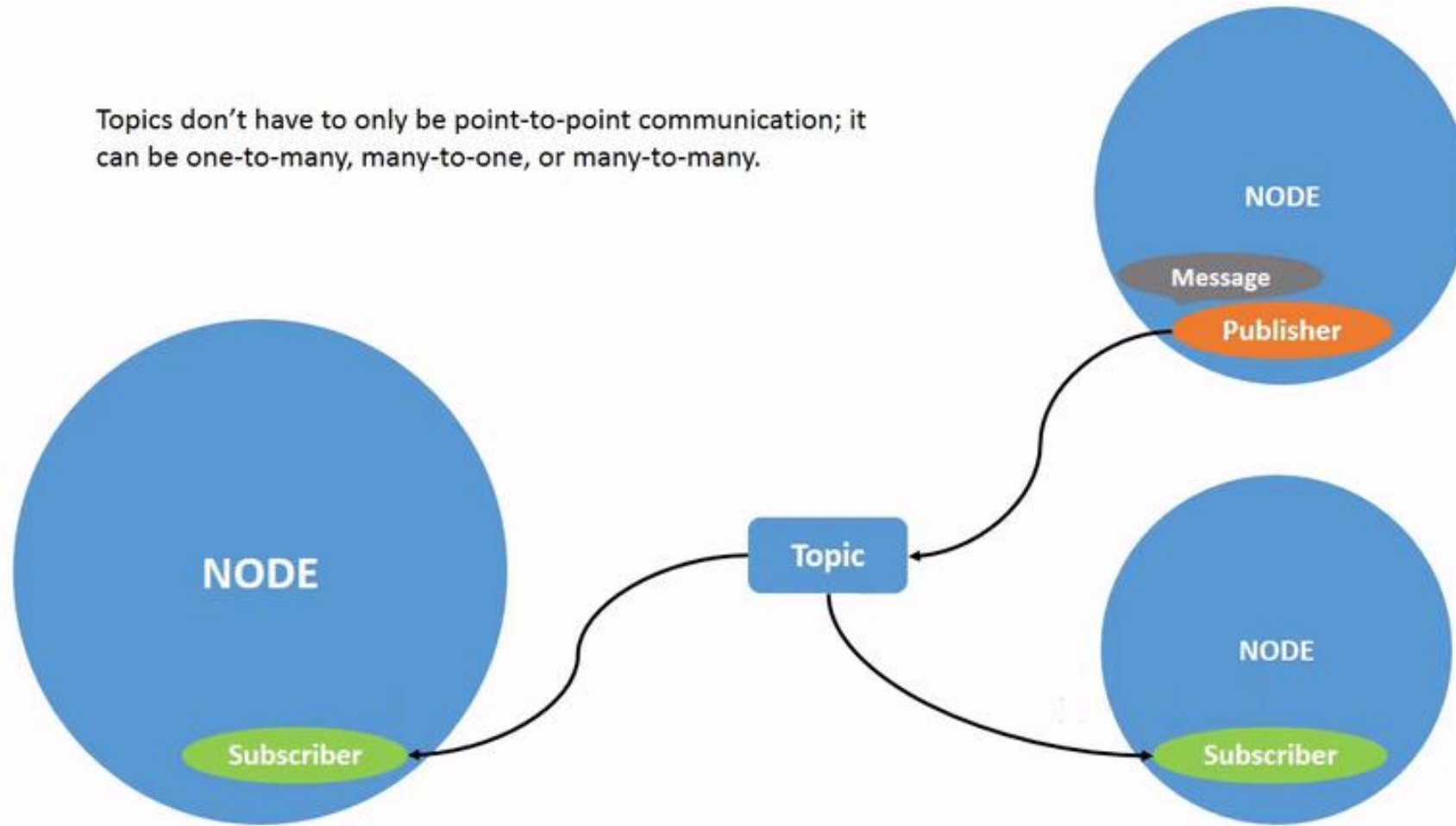
Topics are elements of the ROS graph that act as a bus for nodes to exchange messages.

- Implement the publish/subscribe paradigm
- Can specify Quality of Service (QoS)
- Have a specific message type (int16, String, ...)
- Multiple nodes can publish messages on a topic
- Multiple nodes can read messages from a topic



# Topics

Topics don't have to only be point-to-point communication; it can be one-to-many, many-to-one, or many-to-many.



# Topics

## Example:

Terminal 1:

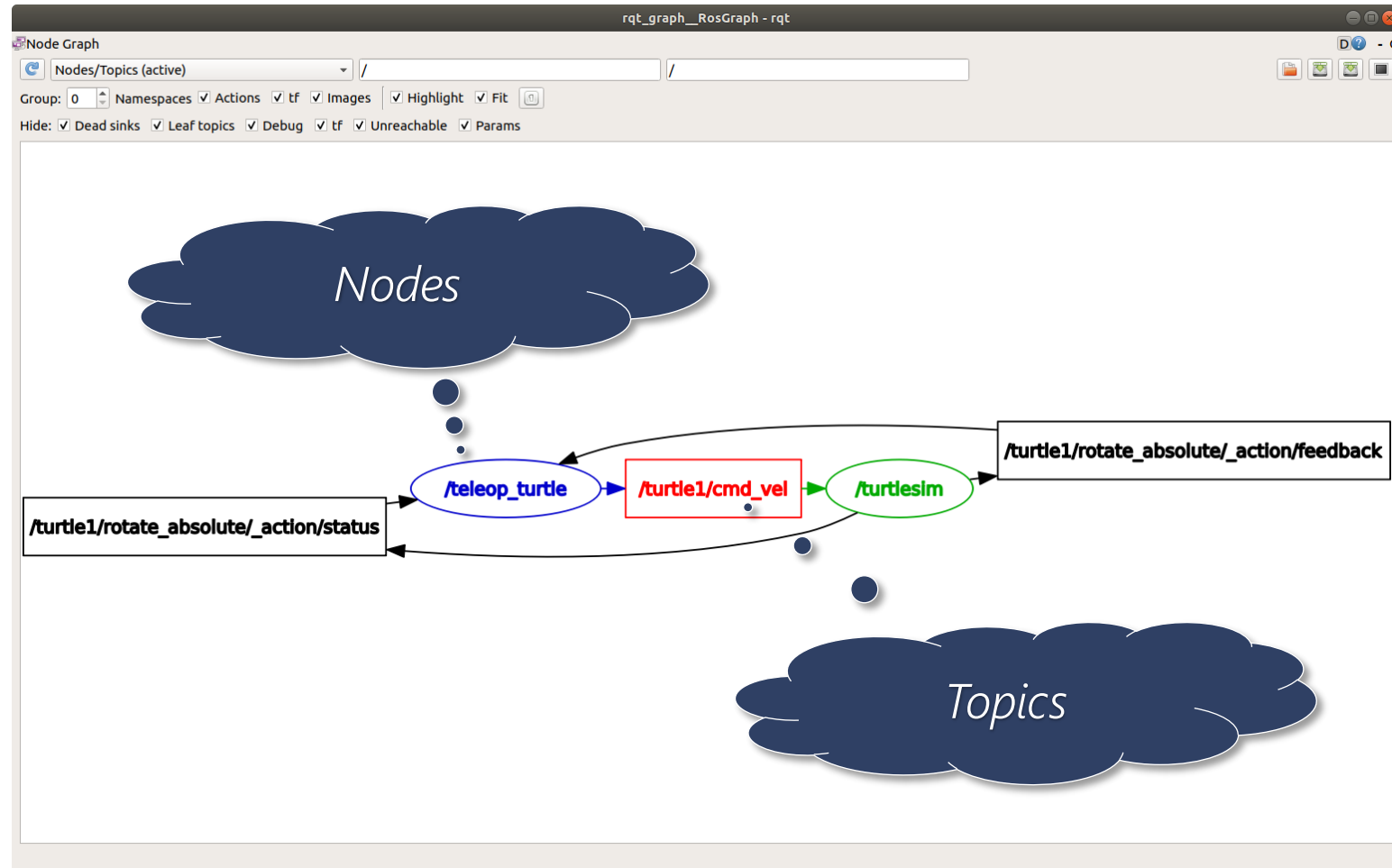
```
ros2 run turtlesim turtlesim_node
```

Terminal 2:

```
ros2 run turtlesim turtle_teleop_key
```

Terminal 3:

```
ros2 run rqt_graph rqt_graph
```





# Messages

Messages are exchanged on topics

- They define the type of the topic
- Several message types already available (use existing msgs if possible!!!)
- It is possible to define new messages using a simple language
- Existing message types can be used in new messages together with base types

std\_msgs/Header.msg

uint32 seq  
time stamp  
string frame\_id

std\_msgs/String.msg

string data

sensor\_msgs/Joy.msg

std\_msgs/Header header  
float32[] axes  
int32[] buttons

# Topics CLI tools

## Example:

Lists all currently active topics

```
ros2 topic list
```

Terminal 1 (echo msgs on topic):

```
ros2 topic echo /turtle1/cmd_vel
```

Terminal 2 (manually publish msgs on topic with frequency of 10 Hz):

```
ros2 topic pub -r 10 /cmd_vel geometry_msgs/Twist '{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0,y: 0.0,z: 0.0}}'
```

# Topics CLI tools

`ros2 topic list` – Lists all currently active topics

`ros2 topic info <topic_name>` – Displays publishers, subscribers, message type, and QoS

`ros2 topic type <topic_name>` – Shows the message type used by a topic

`ros2 topic echo <topic_name>` – Prints messages published on a topic in real time

`ros2 topic pub <topic_name> <msg_type> '<args>'` – Publishes a message to a topic

`ros2 topic bw <topic_name>` – Shows the bandwidth used by a topic

`ros2 topic hz <topic_name>` – Measures and prints the message publishing rate

# How to create a publisher

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html#write-the-publisher-node>

```
class MinimalPublisher(Node):
```

```
    def __init__(self):
```

```
        super().__init__('minimal_publisher')
```

```
        self.publisher_ = self.create_publisher(String, 'topic', 10)
```

```
        ...
```

```
    def timer_callback(self):
```

```
        ...
```

```
        self.publisher_.publish(msg)
```

```
        ...
```

```
def main(args=None):
```

```
    rclpy.init(args=args)
```

```
    minimal_publisher = MinimalPublisher()
```

```
    rclpy.spin(minimal_publisher)
```



```
import rclpy
from rclpy.node import Node

from std_msgs.msg import String

class MinimalPublisher(Node):

    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        timer_period = 0.5 # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello World: %d' % self.i
        self.publisher_.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1

def main(args=None):
    rclpy.init(args=args)

    minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_publisher)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_publisher.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

# How to create a subscriber

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html#write-the-subscriber-node>

```
class MinimalSubscriber(Node):
```

```
    def __init__(self):
```

```
        super().__init__('minimal_subscriber')
```

```
        self.subscription = self.create_subscription(String, 'topic', self.listener_callback, 10)
```

```
    def listener_callback(self, msg):
```

```
        self.get_logger().info('I heard: "%s"' % msg.data)
```

Callback!!!

```
def main(args=None):
```

```
    rclpy.init(args=args)
```

```
    minimal_subscriber = MinimalSubscriber()
```

```
    rclpy.spin(minimal_subscriber)
```

```
import rclpy
from rclpy.node import Node

from std_msgs.msg import String

class MinimalSubscriber(Node):

    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic',
            self.listener_callback,
            10)
        self.subscription # prevent unused variable warning

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)

def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = MinimalSubscriber()

    rclpy.spin(minimal_subscriber)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    minimal_subscriber.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```



# Services

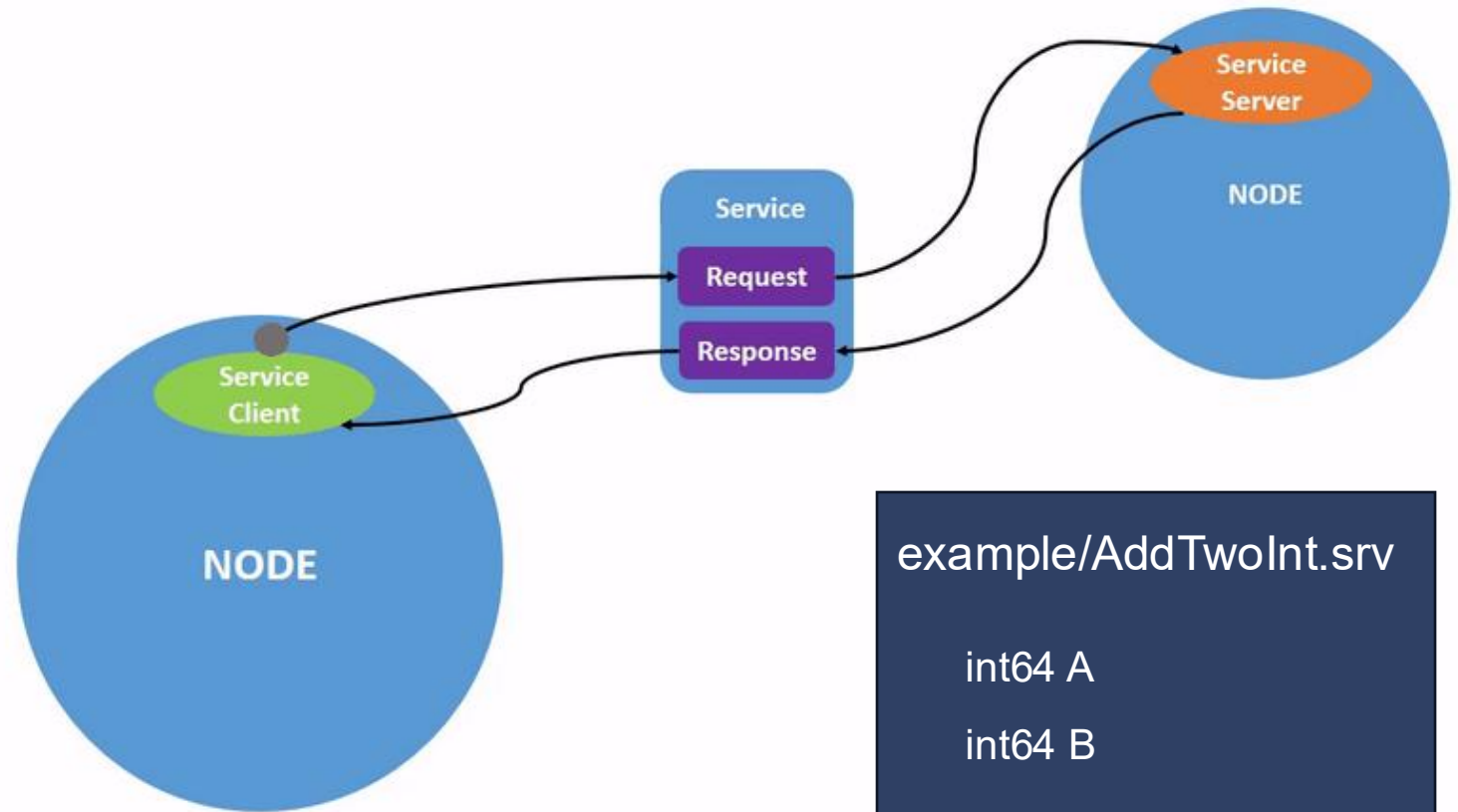
Work like remote function calls

- Implement the **client/server paradigm**
- Code waits for service call to complete
- Guarantee of execution (or report failure)

Slower than topics

(typically > 1ms communication overhead)

→ Avoid services for data streaming



example/AddTwoInt.srv

int64 A

int64 B

---

int64 Sum

# Services CLI tools

`ros2 service list` – Lists all currently available services

`ros2 service info <service_name>` – Displays the service type, nodes providing it, and nodes calling it

`ros2 service type <service_name>` – Displays the message type used by the service

`ros2 service call <service_name> <msg_type> '<args>'` – Sends a request to the service and prints the response

# How to create a service server

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Service-And-Client.html#write-the-service-node>

```
class MinimalService(Node):
```

```
    def __init__(self):
```

```
        super().__init__('minimal_service')
```

```
        self.srv = self.create_service(AddTwoInts,
```

```
                                     'add_two_ints',
```

```
                                     self.add_two_ints_callback)
```

Srv type

Srv name

Callback!!!

```
    def add_two_ints_callback(self, request, response):
```

```
        response.sum = request.a + request.b
```

```
        self.get_logger().info('Incoming request\na: %d b: %d' % (request.a, request.b))
```

```
        return response
```

```
from example_interfaces.srv import AddTwoInts

import rclpy
from rclpy.node import Node

class MinimalService(Node):

    def __init__(self):
        super().__init__('minimal_service')
        self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.add_two_ints_callback)

    def add_two_ints_callback(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info('Incoming request\na: %d b: %d' % (request.a, request.b))

        return response

def main():
    rclpy.init()

    minimal_service = MinimalService()

    rclpy.spin(minimal_service)

    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

# How to create a service client

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Service-And-Client.html#write-the-client-node>

```
class MinimalClientAsync(Node):
    def __init__(self):
        super().__init__('minimal_client_async')
        self.cli = self.create_client(AddTwoInts, 'add_two_ints')
        ...
def main():
    rclpy.init()
    minimal_client = MinimalClientAsync()
    req = AddTwoInts.Request()
    req.a = 1
    req.b = 5
    future = minimal_client.cli.call_async(req)
    rclpy.spin_until_future_complete(minimal_client, future)
    response = future.result()
    minimal_client.get_logger().info('Result of add_two_ints: response.sum')
```

Srv type

Srv name

```
from example_interfaces.srv import AddTwoInts

import rclpy
from rclpy.node import Node

class MinimalService(Node):
    def __init__(self):
        super().__init__('minimal_service')
        self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.add_two_ints_callback)

    def add_two_ints_callback(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info('Incoming request\na: %d b: %d' % (request.a, request.b))

        return response

def main():
    rclpy.init()

    minimal_service = MinimalService()

    rclpy.spin(minimal_service)

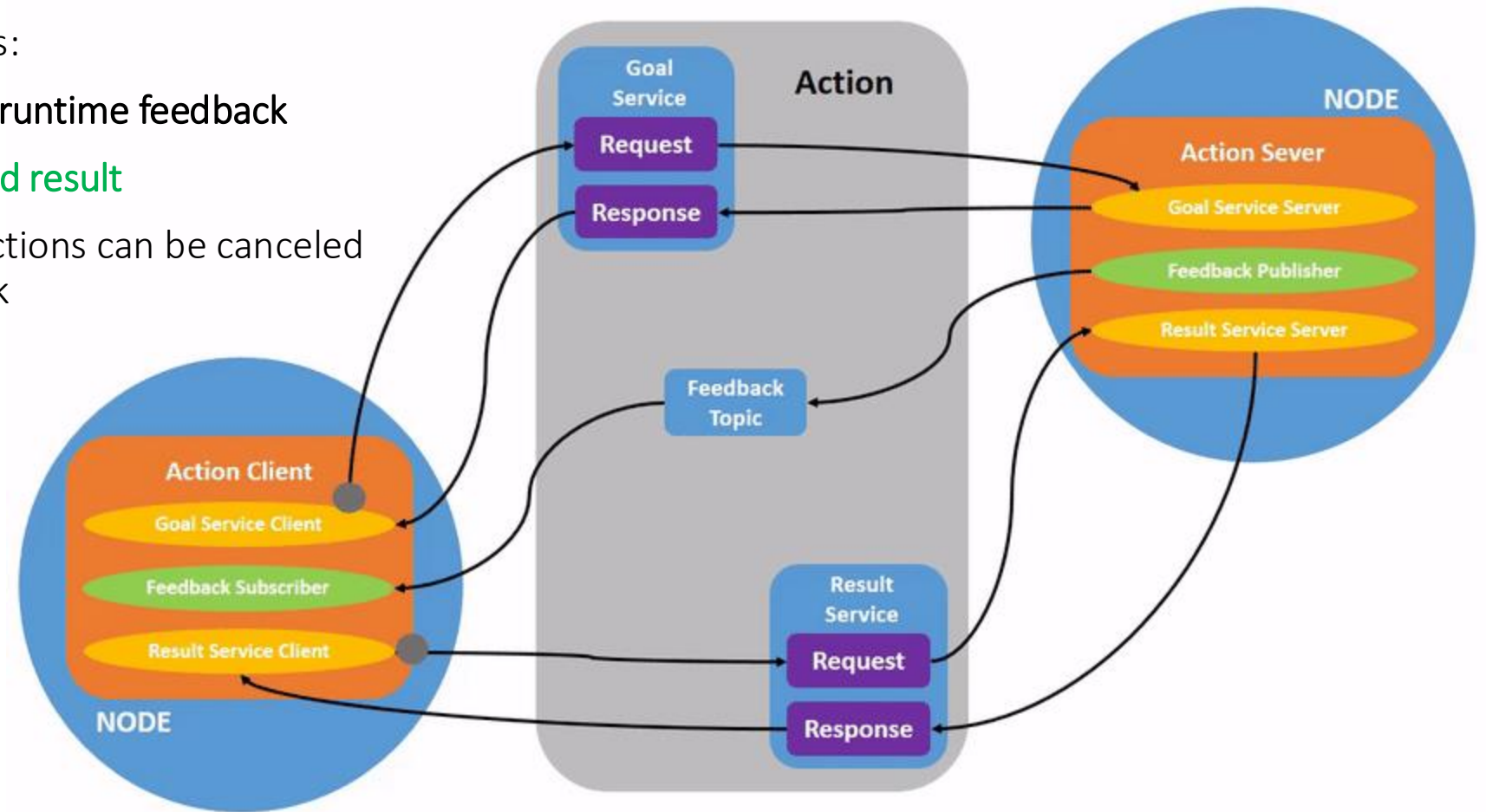
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

# Actions

Intended for long running tasks:

- **Server/client** paradigm with **runtime feedback**
- Consist of **goal, feedback, and result**
- Similar to services, except actions can be canceled and return runtime feedback





# Actions CLI tools

`ros2 action list` – Lists all currently available actions

`ros2 action info <action_name>` – Displays the action type, nodes providing it, and nodes interacting with it

`ros2 action type <action_name>` – Displays the message type used by the action

`ros2 action send_goal <action_name> <msg_type> '<args>'` – Sends a goal request to the action and prints feedback/results

# Launchers

It is often useful to create one or more **launch files to run more nodes at the same time.**

This allows you to

- start all the nodes of the project together
- set all the specified parameters

**Launchers** should be located **in the launch folder** inside your pkg.

Launchers can be written in Python, XML, or YAML.

**NOTE:** Python launchers should be named `<foo>_launch.py` OR `<foo>.launch.py`

# Launchers

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            namespace='turtlesim1',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='turtlesim',
            namespace='turtlesim2',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='turtlesim',
            executable='mimic',
            name='mimic',
            remappings=[
                ('/input/pose', '/turtlesim1/turtle1/pose'),
                ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
            ]
        )
    ])
])
```

Instantiate node of type  
'turtlesim\_node' from pkg  
'turtlesim'

Instantiate another node of type  
'turtlesim\_node' from pkg  
'turtlesim' (under different  
namespace)

ros2 launch <package\_name> <launcher\_name>

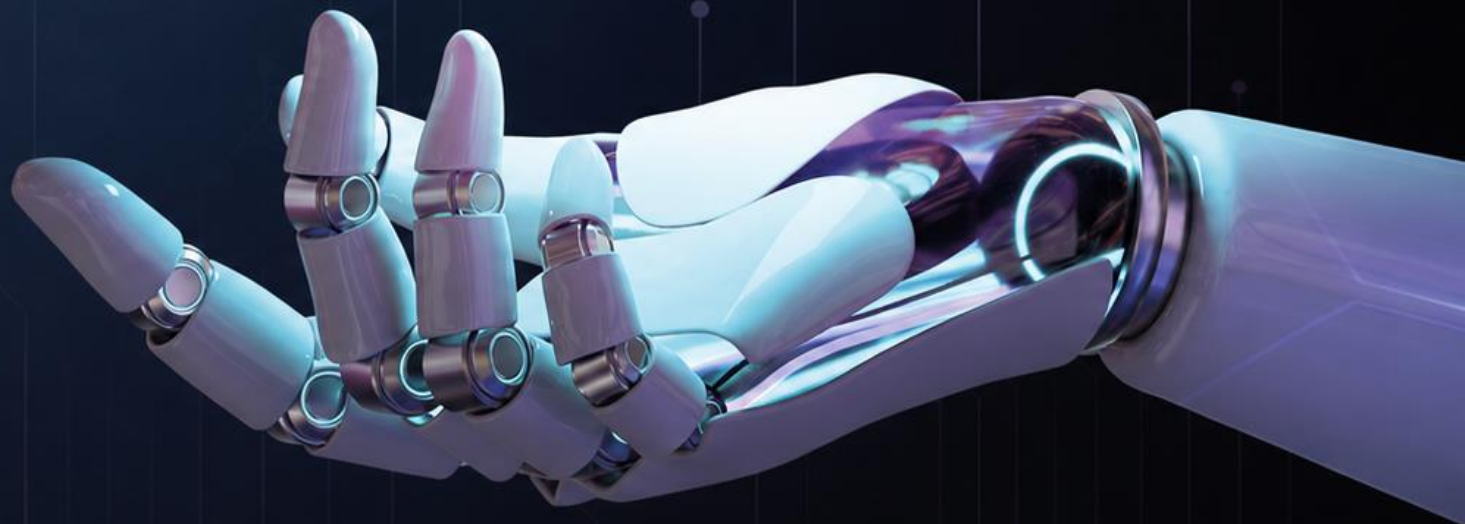
ros2 launch turtlesim turtlesim\_mimic\_launch.py

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Creating-Launch-Files.html>

# Things we did not cover

- [Parameters](#) and [how to set them from an external file](#)
- [Recording and playing back data](#)
- [How to create a workspace from scratch](#)
- [How to create custom msgs and srvs](#)

# ROS2 x DRIMS



Aug. 30 – Sept. 6, 2026  
Poggio all'Agnello (Italy)



Cofinanziato  
dall'Unione europea



Ministero delle Imprese  
e del Made in Italy



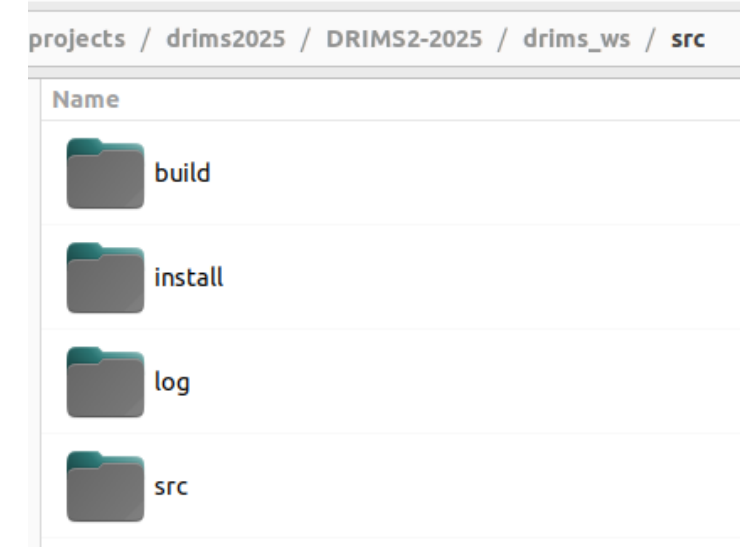


# Workspace structure

A ROS 2 **workspace** is just a directory with 3 main folders created by **colcon build**:

All your stuff  
goes here!

1. **src/** – Source code
  - Where you put your ROS 2 packages (nodes, launch files, etc.)
  - Here you create your packages (with `ros2 pkg create`) or clone existing repos (`git clone`)
2. **build/** – Build artifacts
  - Temporary files from compilation (CMake, intermediate objects, logs)
  - Created automatically, usually not touched by the user
3. **install/** – Install space
  - Final executables, libraries, and setup files
  - What you actually source to use the workspace (`source install/setup.bash`)
  - Created automatically, usually not touched by the user



# Build the workspace

cd drims\_ws directory, then:

## colcon build

Builds all packages in the workspace (in dependency order)

Generates: build/, install/, log/ directories

## colcon build --packages-select <pkg\_name>

Builds only the specified package (and its dependencies)

Useful for faster iteration

## colcon build --symlink-install

Creates symlinks instead of copies between src and install

Lets you modify Python code, YAML files and launchers without rebuilding  
(if you change C++ code you still need to rebuild)

## rm -rf build/ install/ log/

Clean build (remove build/ install/ and log/ folders)

You can also do it manually

*Highly  
recommended!*

# Build the workspace

cd drims\_ws directory, then:

## colcon build

Builds all packages in the workspace (in dependency order)

Generates: build/, install/, log/ directories

## colcon build --packages-select <pkg\_name>

Builds only the specified package (and its dependencies)

Useful for faster iteration

## colcon build --symlink-install

Creates symlinks instead of copies between src and install

Lets you modify Python code, YAML files and launchers without rebuilding  
(if you change C++ code you still need to rebuild)

## rm -rf build/ install/ log/

Clean build (remove build/ install/ and log/ folders)

You can also do it manually

## SOURCE THE WS

```
cd drims_ws  
source install/setup.bash
```

Always\* source the ws after building or whenever you open a new terminal.

You can add this command to your .bashrc file:

```
source ~/drims_ws/install/setup.bash
```

\*not strictly necessary if you don't add new pkgs, but still a good rule of thumb

# Package structure

## C++

|                                      |                                  |
|--------------------------------------|----------------------------------|
| <code>my_cpp_pkg/</code>             |                                  |
| ├── <code>CMakeLists.txt</code>      | ← Build instructions             |
| ├── <code>package.xml</code>         | ← Package metadata               |
| ├── <code>include/my_cpp_pkg/</code> | ← C++ headers (.h, .hpp)         |
| ├── <code>src/</code>                | ← C++ source files (.cpp)        |
| ├── <code>launch/</code>             | ← Launch files (optional)        |
| └── <code>config/</code>             | ← Parameters, configs (optional) |

## Python

|                                  |                                   |
|----------------------------------|-----------------------------------|
| <code>my_py_pkg/</code>          |                                   |
| ├── <code>package.xml</code>     | ← Package metadata                |
| ├── <code>setup.py</code>        | ← Build/install instructions      |
| ├── <code>resource/</code>       | ← Marker file for ament           |
| ├── <code>my_py_pkg/</code>      | ← Python modules (nodes, scripts) |
| │   └── <code>__init__.py</code> | ← Empty file                      |
| ├── <code>scripts/</code>        | ← Executable scripts (optional)   |
| ├── <code>launch/</code>         | ← Launch files (optional)         |
| └── <code>config/</code>         | ← Parameters, configs (optional)  |

ROS code  
goes here!

# Spawn a robot!

Terminal 1:

```
sh start.sh
```

```
ros2 launch drims_description ur5e_1_start.launch.py fake:=true
```

try to moveit from RViz

Terminal 2:

```
sh connect.sh
```

```
ros2 topic echo /joint_states
```

```
ros2 action list
```

# A lot of stuff under the hood

- Where is the **robot geometry**?
  - This includes links, joints, inertias...
- Where are the **controllers**?
  - We should define the interface (position, velocity, effort) to send commands to the robot
  - Along with the actual controller (PID, MPC, ...)
- Where are the **motion planners**?
  - Should include fixed obstacles and the robot geometry (self-collisions)
  - Define the path planner (RRT, PRM,...) and the trajectory planner (spline interpolation, timing law)



# A lot of stuff under the hood

- Where is the robot geometry?
- Where are the controllers?
- Where are the motion planners?

## drims\_summerschool

These are the main repository for the DRIMS Summer School:

IT'S ALL HERE

- [drims\\_cells](#): Contains robot descriptions, controllers, and launch files for setting up simulations and/or establishing communication with real robots.
- [easy\\_motion](#): Provides APIs, ROS actions, and Behavior Tree nodes for basic robot movements.
- [drims\\_dice\\_simulator](#): Offers a simulator for a dice.

For usage instructions and setup details, please refer to each package's README.

[https://github.com/CNR-STIIMA-IRAS/drims\\_summerschool](https://github.com/CNR-STIIMA-IRAS/drims_summerschool)

# Robot description

- ROS2 uses **URDFs** (Universal Robot Description Format) files to describe the robot geometry
- It is convenient to have a **<your\_robot>\_description** pkg that contains the URDF of your cell
- The URDF usually includes other URDFs (e.g., table, robot, camera) to compose a complex environment.
- In our code, see [drims\\_description](#)

# Robot description

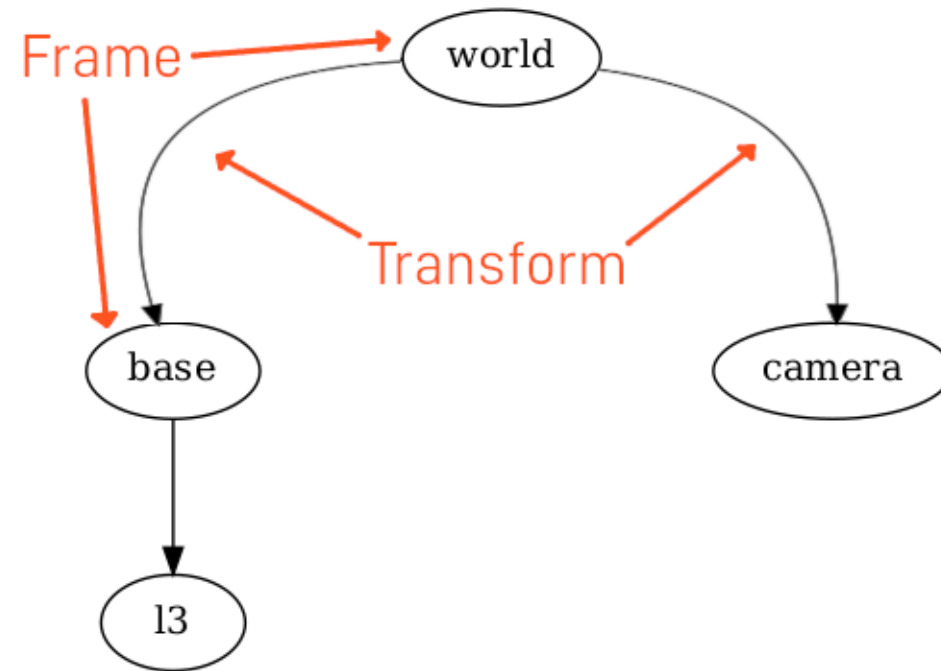
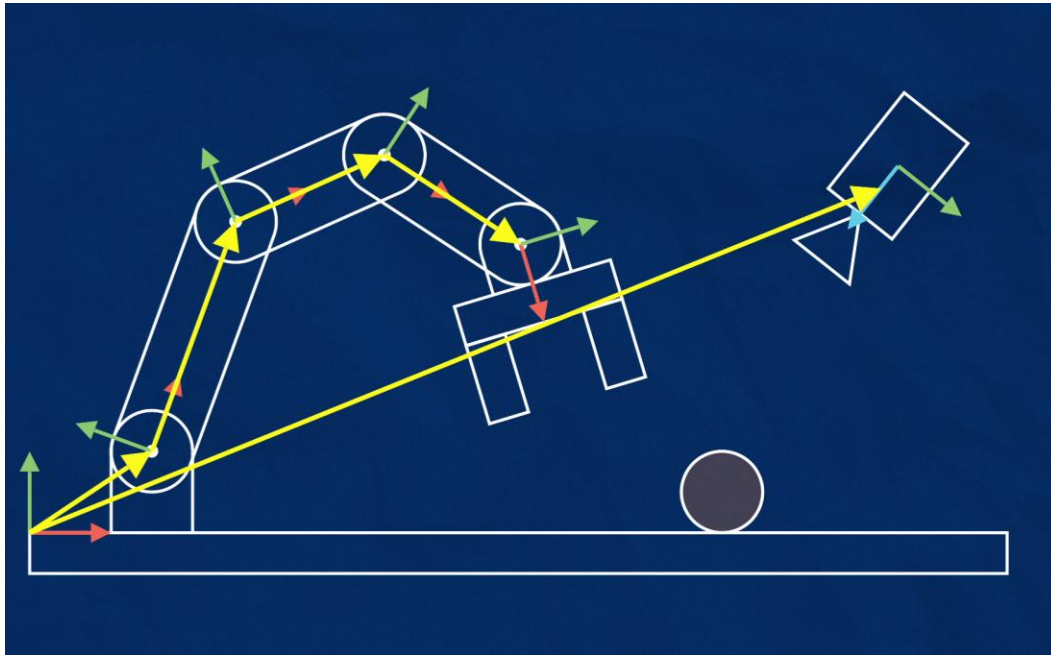
## Example: UR5e + Robotiq Hand-E

- [cell description](#): URDF of the whole cell.
  - It includes the [robot](#) and the gripper's URDFs (provided by the manufacturers)
  - It includes a simplified geometry of the table and virtual walls for safety

```
<xacro:include filename="$(find ur_description)/urdf/ur_macro.xacro"/>
...
<xacro:ur_robot
  name="$(arg name)"
  ...
  use_fake_hardware="$(arg use_fake_hardware)"
  ... >
  <origin xyz="$(table_length/2.0 + 0.1) 0 0" rpy="0 0 0"/>
</xacro:ur_robot>
...
<xacro:include filename="$(find robotiq_hande_description)/urdf/robotiq_hande_gripper.xacro" />
<xacro:robotiq_hande_gripper
  name="robotiq_hande_gripper"
  ...
  use_fake_hardware="$(arg use_fake_hardware)"
/>
```

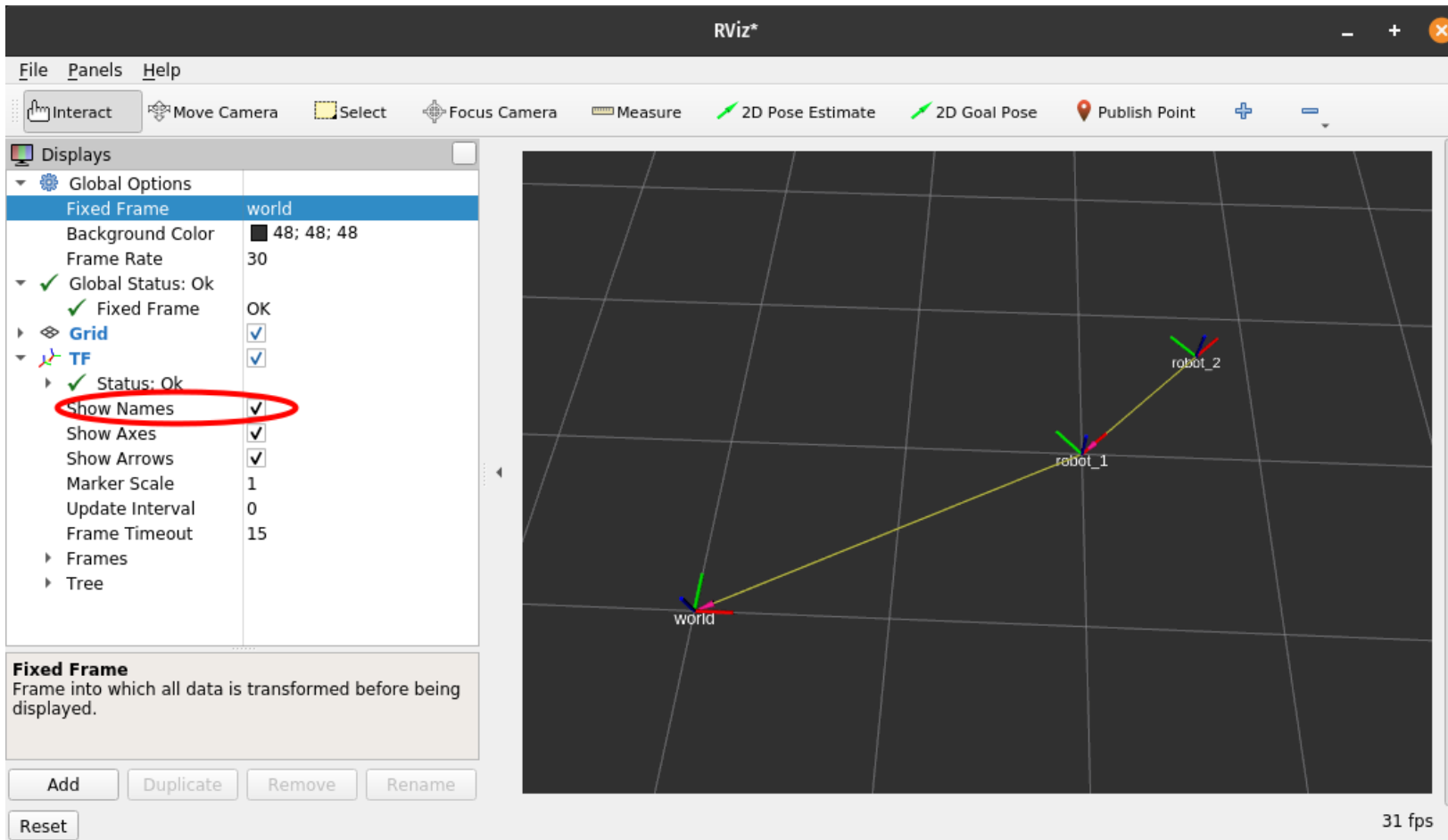
# Transformation matrices (tf)

ROS2 uses the **tf** package to represent affine transformations between frames



```
ros2 run tf2_tools view_frames
ros2 run tf2_ros tf2_echo [source_frame] [target_frame]
```

# Transformation matrices (tf)



## URDF

A tree of...

Links

connected by

Joints

## TF

A tree of...

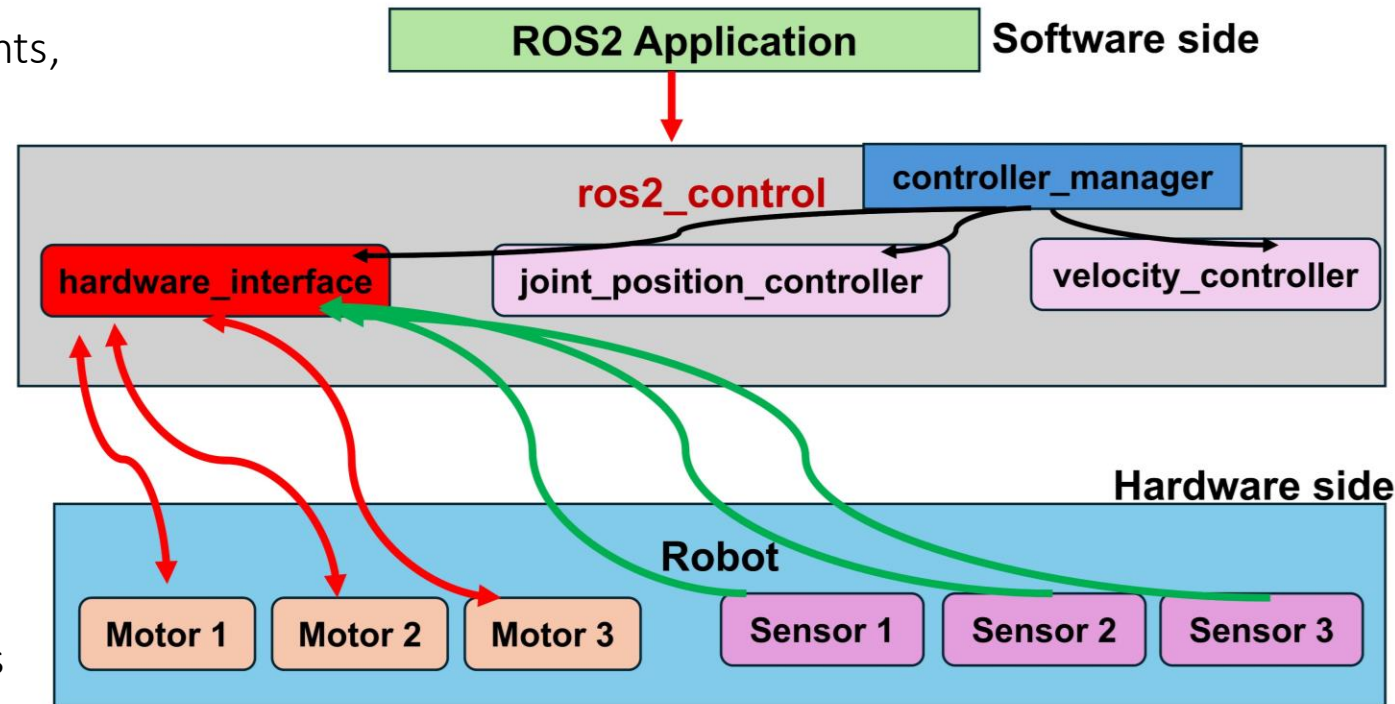
Frames

connected by

Transforms

# Robot control

- [ros2\\_control](#) is the native framework to control robots in ROS2
- **modular** framework for controlling robot hardware (joints, actuators, sensors) in a **standardized** way.
- **Hardware interface (HW interface):**
  - Abstraction layer between ROS 2 controllers and physical hardware.
  - **read()** and **write()** methods to get joint states and send commands from/to the hardware
  - Allows controllers to be **hardware-agnostic**.
- **Controllers:** plugins implementing control logic and communicate with the HW interface.
- **Controller Manager:** a node that loads, starts, and stops controllers at runtime.





# Robot control

Should I Implement low-level control from scratch every time?

**NO!**

- Many robots come with their HW interface
- They already implement basic controllers (position and/or velocity reference)
- If you do **planning** → probably existing controllers are already fine
- If you do **low-level control** → you might need to use / code / modify custom controllers (no need to change the HW interface though)

# Motion planning (MoveIt!)

- ROS2 relies on navigation/motion stacks
- For robot arms, the standard is [MoveIt2](#)
- MoveIt provides **path planners**, **trajectory planners**, **collision checking**
- When you move the robot from Rviz, you are actually using MoveIt

To use MoveIt, you can automatically create a **<your\_robot>\_moveit\_config** pkg that contains the planning configurations

For our UR5e:

- [drims2\\_ur10e\\_moveit\\_config](#)
- [ur5e\\_moveit.launch.py](#)

```
RRT:
  type: geometric::RRT
  range: 0.0 # Max motion added to tree. ==> maxDistance_ default: 0.0, if 0.0, set on setup()
  goal_bias: 0.05 # When close to goal select goal, with this probability? default: 0.05
RRTConnect:
  type: geometric::RRTConnect
  range: 0.0 # Max motion added to tree. ==> maxDistance_ default: 0.0, if 0.0, set on setup()
RRTstar:
  type: geometric::RRTstar
  range: 0.0 # Max motion added to tree. ==> maxDistance_ default: 0.0, if 0.0, set on setup()
  goal_bias: 0.05 # When close to goal select goal, with this probability? default: 0.05
  delay_collision_checking: 1 # Stop collision checking as soon as C-free parent found. default 1
```

# The DRIMS code

[drims\\_summerschool](#) is a hub repo that contains three repositories to spawn and control the robots:

- ✓ [drims\\_cells](#): Contains robot descriptions, controllers, and launch files for setting up simulations and/or establishing communication with real robots.
- [easy\\_motion](#): Provides APIs, ROS actions, and Behavior Tree nodes for basic robot movements.
- [drims\\_dice\\_simulator](#): Provides a lightweight simulator for a dice.

These packages are already installed in the docker image.  
You don't need to download them again.

# Your code

Fork [drims homework](#) from Github



Samuele  
Sandrini



Michele  
Ferrari



Roberto  
Fausti

 CNR-STIIMA-IRAS / [drims\\_homework](#) Public

 Notifications

 Fork 33

 Star 3

 Code  Issues  Pull requests  Actions  Projects  Security and quality  Insights



```
cd drims_ws/src
```

```
git clone https://github.com/<your github user name>/drims_homework.git
```

# Hands on the DRIMS challenge: First steps



- Use the robot to move the die to expose the requested face
- Do it as fast as possible

Hands-on  
Sep 2

## Minimal skills:

1. Use camera to localize the dice and identify the current face
2. Move the robot and actuate the gripper to grasp the dice and re-orient it

Hands-on  
Sep 4

Constraints: robot speed cannot be increased (for safety reasons)

# How to approach the problem?

## Problem:

1. Use camera to localize the dice and identify the current face
2. Move the robot and actuate the gripper to grasp the dice and re-orient it

## N00b solution:

identify dice face

IF current\_face == desired face → SUCCESS

ELSE:

localize dice

plan motion to pose above the dice

execute motion to pose above the dice

plan motion to move linearly and approach the dice

execute motion to move linearly and approach the dice

close the gripper

plan motion to move up

execute motion to move up

plan motion to rotate 90° around x-axis

execute motion to rotate 90° around x-axis

plan motion to move down

execute motion to move down

open gripper

identify dice face

IF current\_face == desired face → SUCCESS

ELSE:

.....



# How to approach the problem?

## Desiderata:

1. The code should be re-usable (possibly also in other similar projects). Avoid code duplication as much as possible!
2. The approach should be robot-independent (minimal changes if we change the robot setup, no maintenance if the robot driver needs to be updated...)
3. Decouple the high-level logic to solve the task and the low-level instructions to move the robot
  1. Allows seamless extension (what if the die has 12 faces?)
  2. High-level logic can easily be made robot independent
  3. Can re-use the low-level skills in other projects

# How to approach the problem?

## Problem:

1. Use camera to localize the dice and identify the current face
2. Move the robot and actuate the gripper to grasp the dice and re-orient it

## N00b solution:

identify dice face

IF current\_face == desired face → SUCCESS

ELSE:

localize dice

plan motion to pose above the dice

execute motion to pose above the dice

plan motion to move linearly and approach the dice

execute motion to move linearly and approach the dice

close the gripper

plan motion to move up

execute motion to move up

Most operations are very similar,  
they just differ in the parameters!

plan motion to rotate 90° around x-axis

execute motion to rotate 90° around x-axis

plan motion to move down

execute motion to move down

open gripper

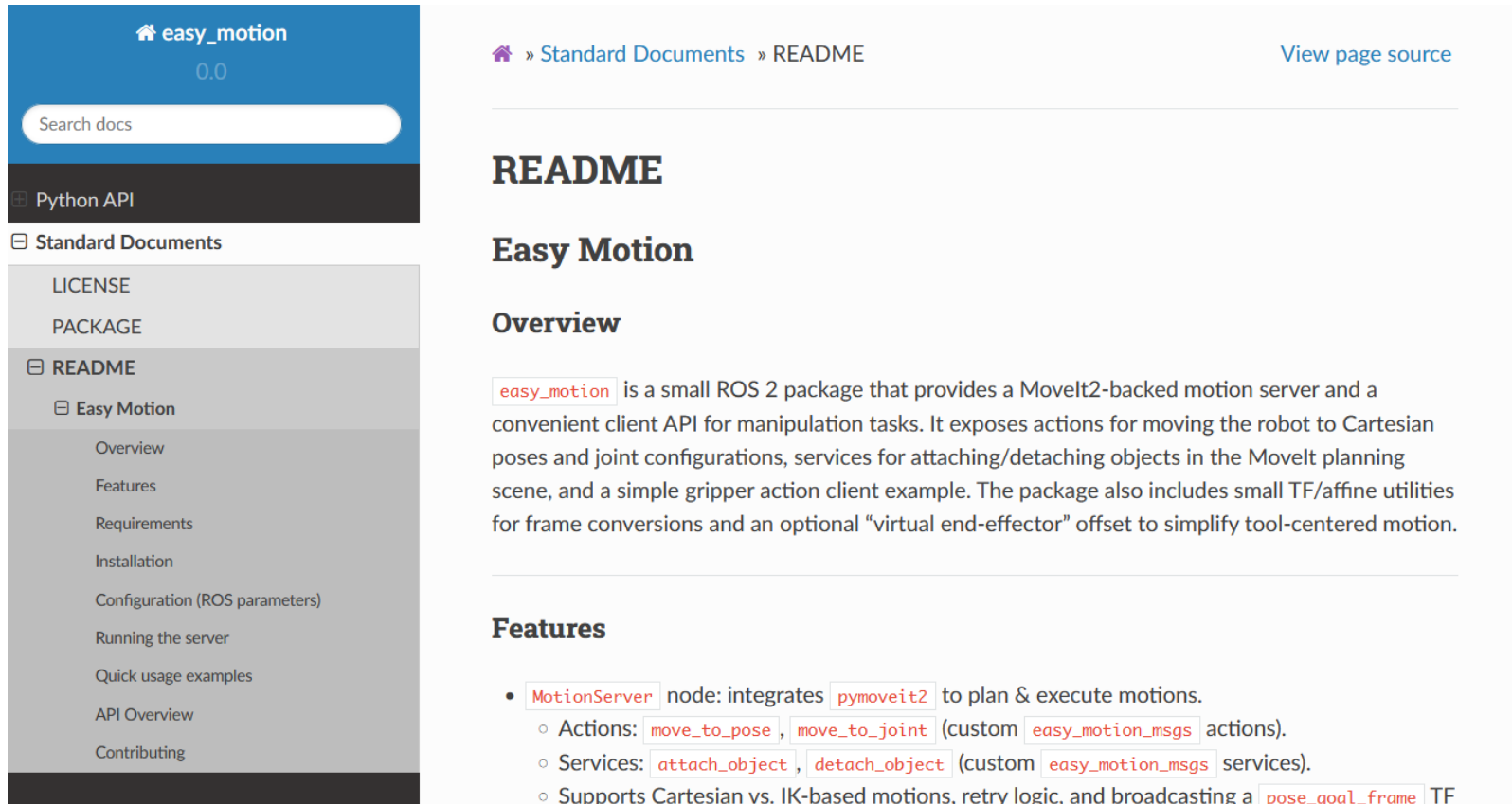
identify dice face

IF current\_face == desired face → SUCCESS

Calling MoveIt may require a  
dozen lines of code every time

# easy\_motion: motion abstraction layer

We have already done most of the work: [https://github.com/CNR-STIIMA-IRAS/easy\\_motion](https://github.com/CNR-STIIMA-IRAS/easy_motion)



**easy\_motion** 0.0

Search docs

Python API

Standard Documents

LICENSE

PACKAGE

README

Easy Motion

Overview

Features

Requirements

Installation

Configuration (ROS parameters)

Running the server

Quick usage examples

API Overview

Contributing

» Standard Documents » README [View page source](#)

## README

### Easy Motion

#### Overview

`easy_motion` is a small ROS 2 package that provides a MoveIt2-backed motion server and a convenient client API for manipulation tasks. It exposes actions for moving the robot to Cartesian poses and joint configurations, services for attaching/detaching objects in the MoveIt planning scene, and a simple gripper action client example. The package also includes small TF/affine utilities for frame conversions and an optional “virtual end-effector” offset to simplify tool-centered motion.

#### Features

- `MotionServer` node: integrates `pymoveit2` to plan & execute motions.
  - Actions: `move_to_pose`, `move_to_joint` (custom `easy_motion_msgs` actions).
  - Services: `attach_object`, `detach_object` (custom `easy_motion_msgs` services).
  - Supports Cartesian vs. IK-based motions, retry logic, and broadcasting a `pose_goal_frame` TF



Samuele Sandrini

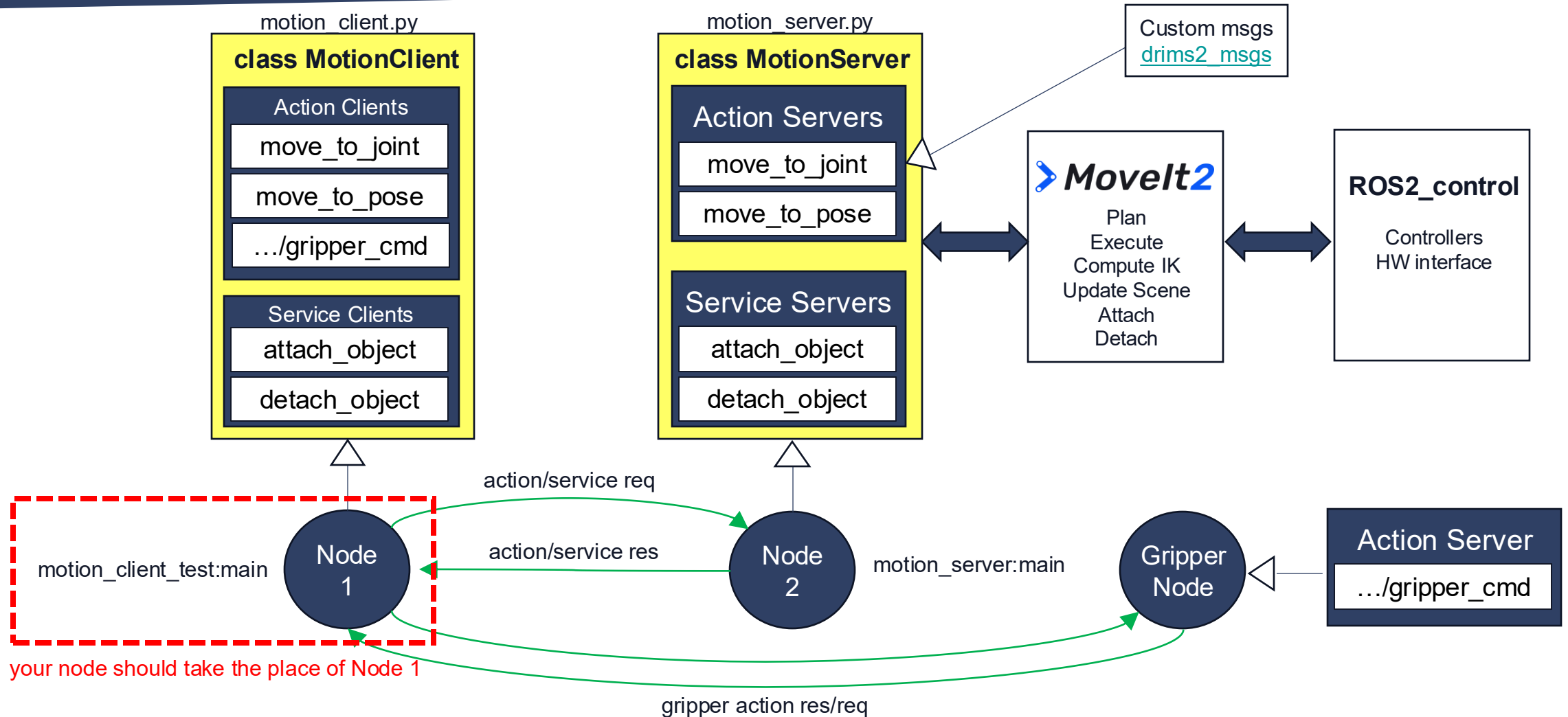
# *easy\_motion*: motion abstraction layer

You should directly use *easy\_motion*'s APIs to move the robot.

From python:

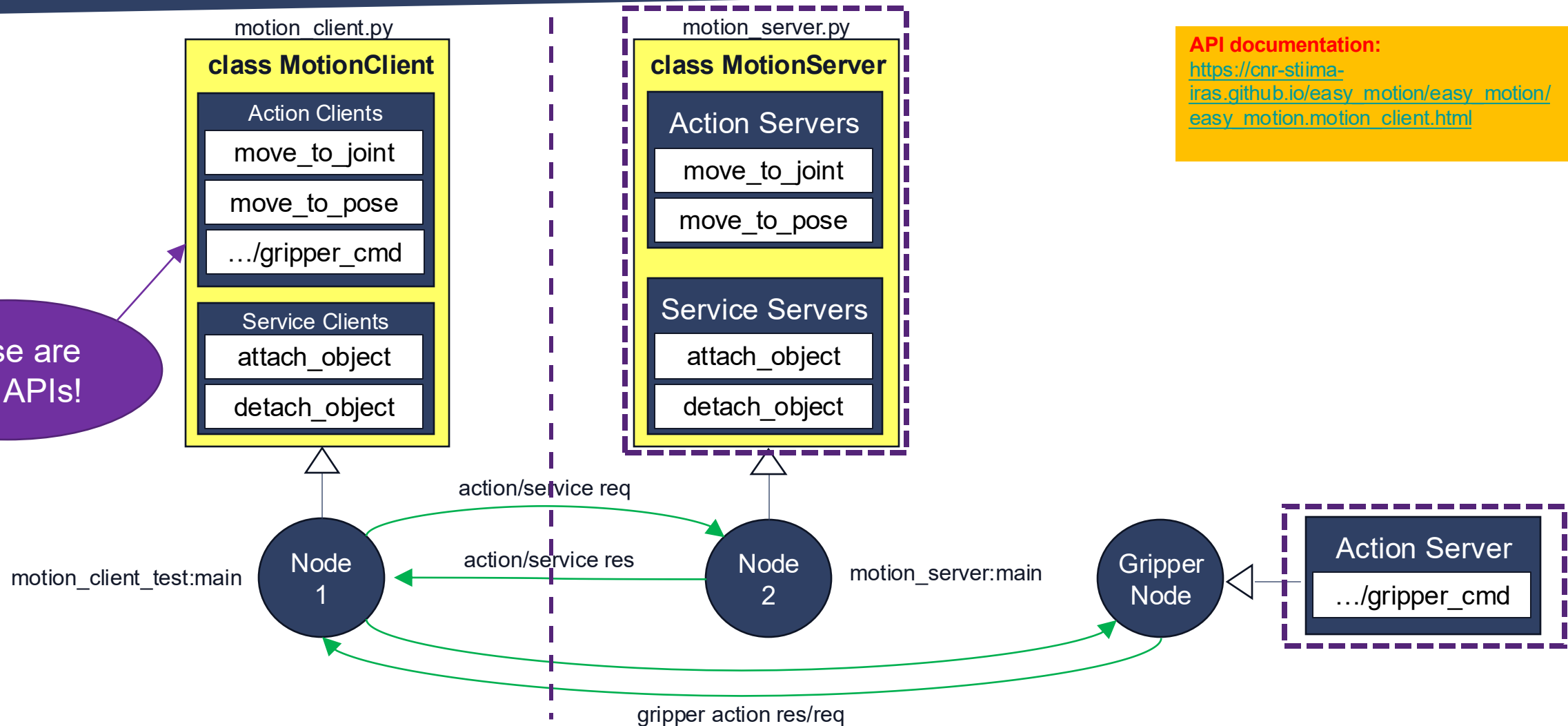
- `move_to_joint(...)`
- `move_to_pose(...)`
- `plan_to_joint(...)`
- `plan_to_pose(...)`
- `get_ik(...)`
- `get_fk(...)`
- ... see [https://cnr-stiima-iras.github.io/easy\\_motion/easy\\_motion/easy\\_motion.motion\\_client.html](https://cnr-stiima-iras.github.io/easy_motion/easy_motion/easy_motion.motion_client.html)

# Available software architecture



# Available software architecture

These are  
your APIs!



**API documentation:**

[https://cnr-stiima-  
iras.github.io/easy\\_motion/easy\\_motion/  
easy\\_motion.motion\\_client.html](https://cnr-stiima-iras.github.io/easy_motion/easy_motion/easy_motion.motion_client.html)



# Run the code

Terminal 1: Spawn the robot

```
ros2 launch drims_description ur5e_1_start.launch.py fake:=true
```

Terminal 2: Spawn simulated dice

```
ros2 launch drims_dice_simulator spawn_dice.launch.py face_up:=5 position:="[-0.1, 0.0, 0.85]"
```

Terminal 3: Run example

```
ros2 action list
```

```
ros2 launch drims_homework demo_python_node_start.launch.py
```

Only for simulation. Will be replaced by your vision algorithm

Inspect available actions and services!!

# How to approach the problem?

## Desiderata:

1. The code should be re-usable (possibly also in other similar projects). Avoid code duplication as much as possible!
2. The approach should be robot-independent (minimal changes if we change the robot setup, no maintenance if the robot driver needs to be updated...)
3. **Decouple the high-level logic to solve the task and the low-level instructions to move the robot**
  1. Allows seamless extension (what if the die has 12 faces?)
  2. High-level logic can easily be made robot independent
  3. Can re-use the low-level skills in other projects

# How to approach the problem?

## Problem:

1. Use camera to localize the die and identify the current face
2. Move the robot and actuate the gripper to grasp the die and re-orient it

The problem can be solved by calling these 'skills' multiple times in the right order

**Skill A:** Move the robot to a given pose (e.g., above the die)

**Skill B:** Move the robot end-effector incrementally in Cartesian space (e.g., to approach die)

**Skill C:** Open/close the gripper

**Skill D:** Process camera images to localize the die and understand which face is exposed

Define a state machine to combine the skills

# High-level logic

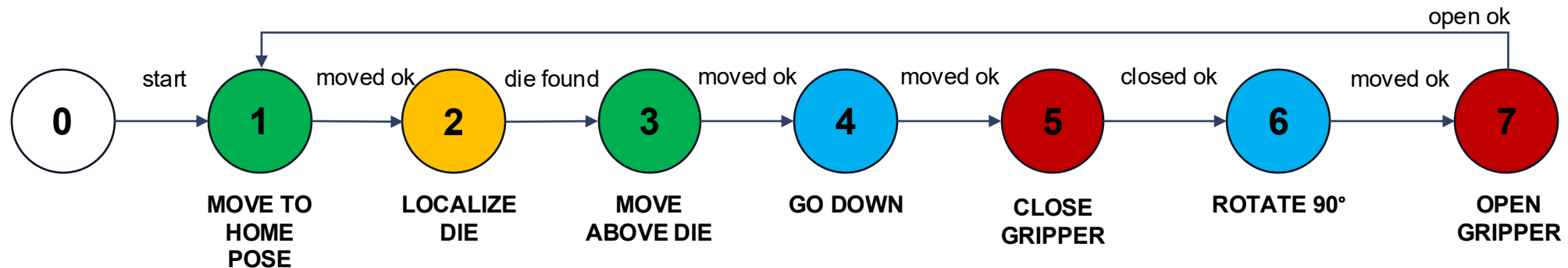
**Skill A:** Move the robot to a given pose (e.g., above the die)

**Skill B:** Move the robot end-effector incrementally in Cartesian space (e.g., to approach the die)

**Skill C:** Open/close the gripper

**Skill D:** Process camera images to localize the dice and understand which face is exposed

Define a state machine to combine the skills



# High-level logic

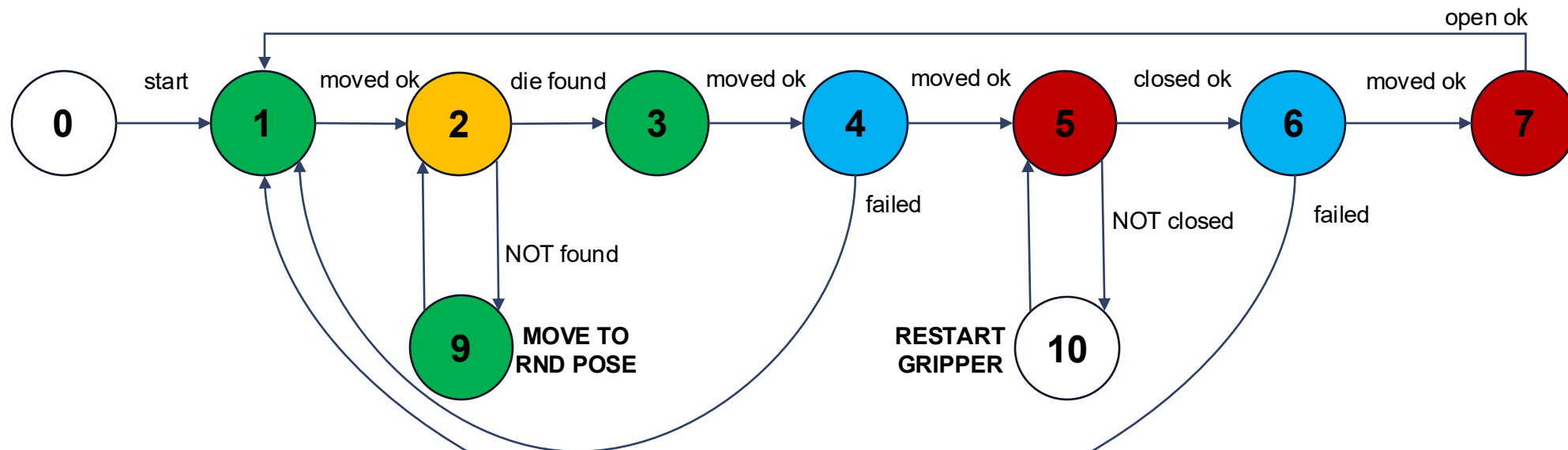
**Skill A:** Move the robot to a given pose (e.g., above the die)

**Skill B:** Move the robot end-effector incrementally in Cartesian space (e.g., to approach the die)

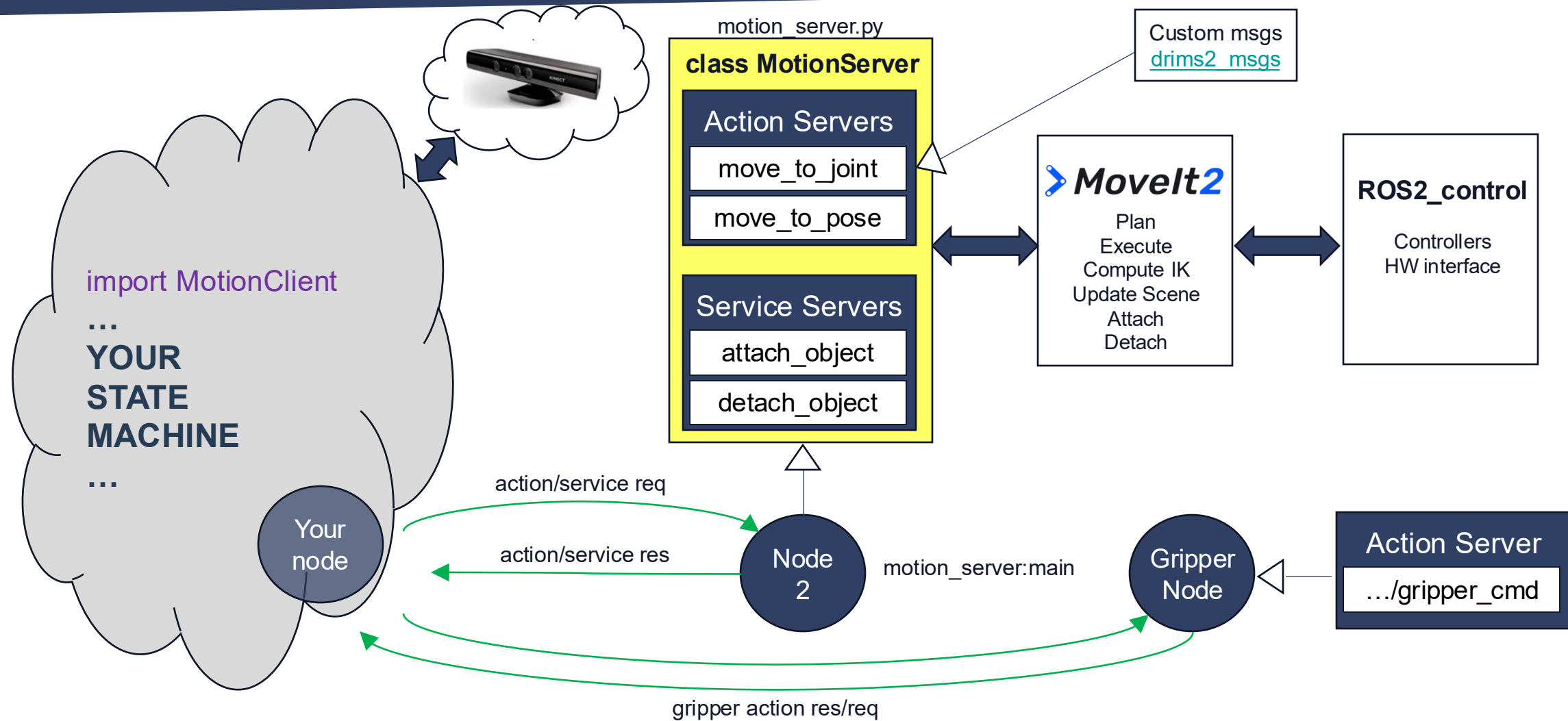
**Skill C:** Open/close the gripper

**Skill D:** Process camera images to localize the dice and understand which face is exposed

Define a state machine to combine the skills

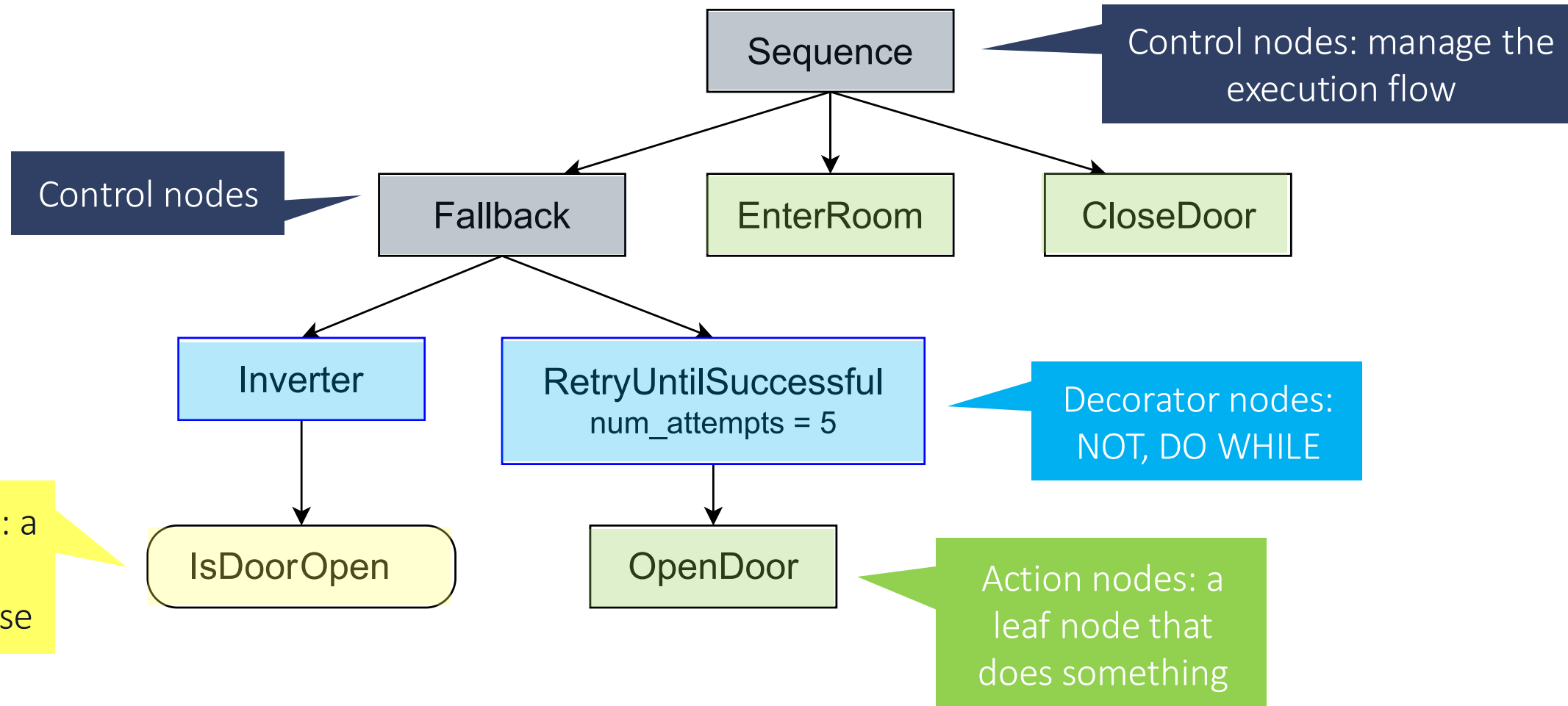


# Back to the challenge





# A nice alternative: Behavior Trees



# BT: An example

Terminal 1: Spawn the robot

```
ros2 launch drims_description ur5e_1_start.launch.py fake:=true
```

Terminal 2: Spawn simulated dice

```
ros2 launch drims_dice_simulator spawn_dice.launch.py face_up:=5 position:=[-0.1, 0.0, 0.85]
```

Terminal 3: Run example

```
ros2 launch drims_homework demo_behavior_tree_start.launch.py
```

# Homework

Inspect the code in [drims\\_homework](#)

Understand what's going on in [demo\\_python\\_node\\_start.launch.py](#)

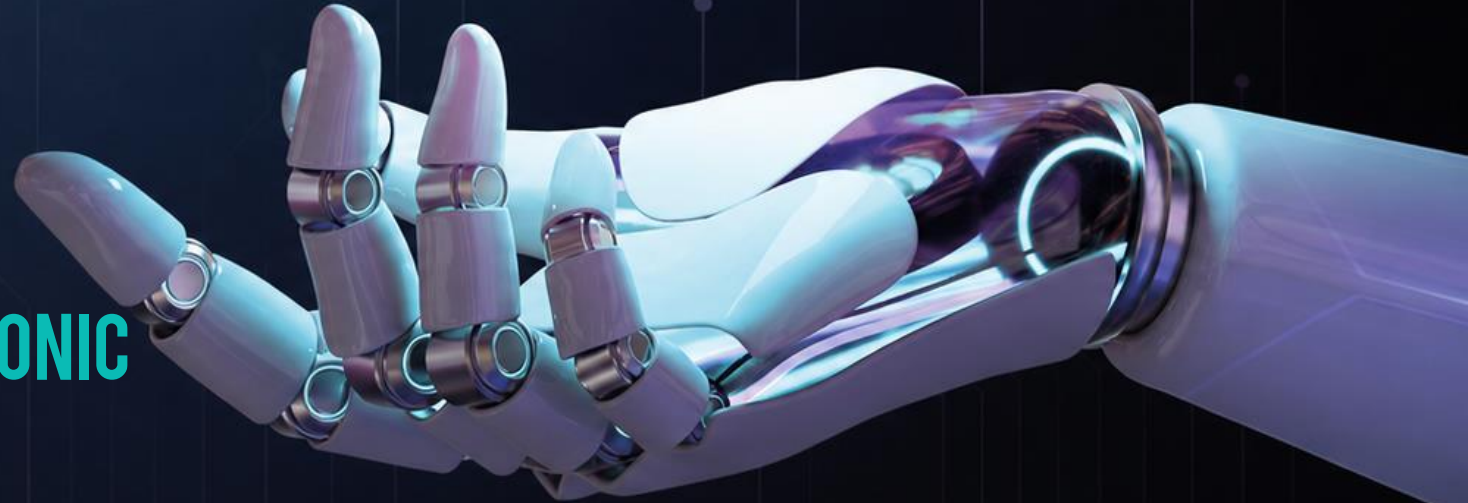
Understand how to use the APIs (e.g., `move_to_pose`)

Optional: Understand what's going on in [demo\\_behavior\\_tree\\_start.launch.py](#)

# Dinner time!

4<sup>th</sup> Doctoral Summer School on Robotics and Intelligent Machines – DRIMS

Gold Sponsor



Silver Sponsor



Bronze Sponsor



Aug. 30 – Sept. 6, 2026  
Poggio all'Agnello (Italy)

